

Flow-Conditioned Metric 3D Point Tracking

Optical flow + metric depth + a DINOv3-conditioned Vision-Mamba-3 depth refiner

Masahiro Ogawa

July 24, 2026

Abstract

We address tracking query points in *metric* 3D across a monocular video, where the quantity that ultimately governs real-world accuracy is the per-point depth. Rather than learning tracking end-to-end, we compose two frozen, well-understood front-ends — dense optical flow [1, 2] for 2D correspondence and a monocular metric-depth network [3] for the third dimension — and learn only the residual they cannot supply: the depth along each point’s (fixed) pixel ray, refined by a tiny causal Vision-Mamba-3 state-space model [4] conditioned on DINOv3 [5] appearance features. This isolates the single degree of freedom that determines 3D error, while keeping the model small (~ 0.5 M parameters) and the pipeline interpretable. On the TAPVid-3D [6] minival benchmark, our best configuration (WAFT flow + Depth-Anything-3 depth + the DINOv3-conditioned refiner) attains the highest *absolute* metric accuracy of all methods evaluated under identical conditions — mean metric-AJ 0.246, ahead of strong feed-forward 3D trackers — confirming that a targeted depth-only refinement recovers real-world 3D structure that scale-invariant training leaves on the table. To measure this we introduce an absolute-metric evaluation protocol that scores error at fixed metre thresholds, exposing depth quality that the standard scale-invariant leaderboard metric hides. A companion reliability analysis — reproduced with each competitor’s own official evaluator — explains why several published 3D trackers collapse under a fixed monocular, single-GPU budget, separating genuine method limitations from evaluation artefacts.

Contents

1	Method	2
1.1	Terminology: Mamba-3 and Vision-Mamba-3	2
1.2	v1–v32: prior direct-regression attempts (prehistory)	2
1.3	v1–v32 architecture: within-frame + across-frame Vision-Mamba-3 (early tracker)	3
1.3.1	Architecture overview	4
1.3.2	Within-frame block: VSSD self-attention	4
1.3.3	Across-frame block: causal cross-SSD	5
1.3.4	Per-track read-out heads	5
1.3.5	Spawn and disappear	6
1.3.6	Loss	6
1.3.7	Why this operator pairing	7
1.4	Overview	8
1.5	2D track: SEA-RAFT flow-chaining (baseline, frozen)	8
1.6	Metric depth and differentiable unprojection	8
1.7	v33: Mamba-3 depth-along-ray refiner (the method)	9
1.8	Why depth-only (and not 2D)	9
1.9	v35: image-conditioned joint 2D–depth refiner	10
1.10	v36: bidirectional flow	12
1.11	v37–v39: WAFT optical-flow front-end	13
1.12	v40: WAFT + DA3-g metric depth (no refiner)	13

1.13	DA3-g re-evaluation of the external SOTA baselines	14
1.14	v41: DA3-g depth + vmamba3 refiner	15
1.15	v41b: pstudio-only retrain (systematic-vs-noise check)	16
1.16	v42: near-field boost by pstudio oversampling	16
1.17	DA3-l vs. DA3-g: architecture, loss, and where the near-field error really is	17
1.18	v43: in-model per-frame scale head	20
1.19	v44: DA3-g de-flicker (standalone)	21
1.20	v45: v44 de-flicker + v35 refiner	22
1.21	v46: world-frame tracking with ego-motion/object-motion decomposition (proposed)	23
1.22	Error analysis and proposed refinements	26
1.23	v47: refinements targeting the localised errors	27
1.24	Training	28
2	Evaluation	28
2.1	Protocol and metrics	28
2.2	Leaderboard (scale-invariant) results	29
2.3	Absolute-metric results: the reversal	30
2.4	Comparison to state-of-the-art	31
2.5	Qualitative results	35
2.6	Efficiency	36

1 Method

1.1 Terminology: Mamba-3 and Vision-Mamba-3

The tracker below is an application of *Vision-Mamba-3*, our reformulation of the Mamba-3 state-space model as a drop-in *attention* operator for vision. Two names recur, and we use them precisely:

- **Mamba-3** — the base Mamba-3 [4] state-space model and its structured state-space duality: the causal 1-D sequence operator we build on. In this note “Mamba-3” also labels a use of our operator on *purely geometric* input (ray, depth-ratio, visibility — no appearance), as in the v33/v44 refiners.
- **Vision-Mamba-3** — our SSD-based *self*- and *cross-attention* operators (`Mamba3SelfAttention` / `Mamba3CrossAttention`), computing $\mathbf{Y} = (\mathbf{L} \odot \mathbf{CB}^\top)\mathbf{V}$ with a *structured decay mask* $\mathbf{L}$ in place of softmax scores — linear in sequence length — applied to *vision* input: learned image/appearance features (DINOv3) or 2-D / multi-view spatial tokens. It runs in causal (streaming, over time) or non-causal (within-frame / multi-view) mode. Every DINOv3-conditioned refiner here (v35 onward) is Vision-Mamba-3.

A module is *Vision-Mamba-3* when it runs this operator on vision input, and *Mamba-3* when on geometry alone. The operator’s full derivation — SSD $\equiv$ linear attention, the self/cross forms, 2-D RoPE, and the non-causal (VSSD) variant — is the companion note `mamba3.attention`.

1.2 v1–v32: prior direct-regression attempts (prehistory)

The freeze-flow-and-refine-depth design below is the product of a long line of direct-regression trackers (v1–v32) that mostly *failed*; those failures motivate it, so we summarise them briefly. (The numbers here are on an earlier per-clip eval protocol and are not directly comparable to the tables in §2.) **v1–v17** trained a *Vision-Mamba-3* tracker to regress 3D tracks end-to-end — from-scratch encoders, then frozen DINOv2/DINOv3 backbones, correlation-volume features

and iterative refinement, with heavy loss engineering (Smooth-L1/Huber + direction-cosine, a 2D reprojection term, cumsum-trajectory and scale-normalised losses); its within-frame/across-frame Vision-Mamba-3 architecture and the v6 loss are detailed in §1.3–§1.3.6. The recurring failure was a *trivial zero-motion collapse*: the network minimised the loss by predicting near-static tracks. **v18–v28** attacked the metric-*scale* problem head-on with a predicted per-clip scale head and a battery of scale-invariant / path-length-normalised losses and ground-truth-scale definitions (RMS motion vs. median anchor depth); trained on the full TAPVid-3D set (v19) this plateaued at $\approx 8\%$ mean 3D-AJ. **v29–v31** scaled the backbone (DINOv3-L, 300 M frozen) and added a DA3-Large metric-depth branch, but v31 collapsed again (0.5% mean 3D-AJ).

The turning point was **v32**: freeze SEA-RAFT flow and DA3 depth, and learn only a tiny (0.4 M) causal Mamba-3 that refines the 2D track with a $\Delta \mathbf{u}$ head before unprojection. It cured the collapse but gave a clean *negative* result — the learned $\Delta \mathbf{u}$ *hurt* 3D-AJ on every subset versus the training-free SEA-RAFT+DA3 baseline (mean 5.2% $\rightarrow$ 3.4%), because nudging $\mathbf{u}$ across DA3 depth discontinuities flips points out of the metric threshold. That single observation is the crux of the method: *do not move the 2D track at all* — freeze it and refine only the depth along the fixed ray. That is v33 (§1.7) — the first version to beat the baseline.

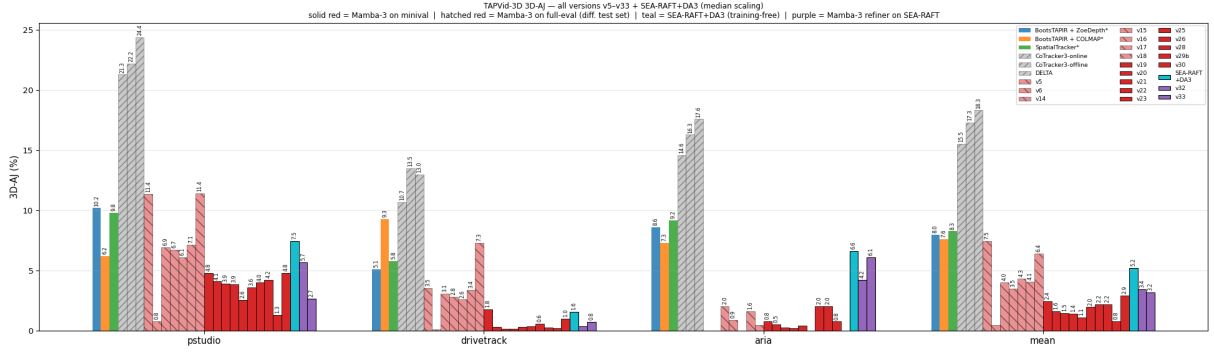


Figure 1: TAPVid-3D 3D-AJ (median-scaled, %) for the whole v1–v32 prehistory, per subset and mean. Hatched red = end-to-end Vision-Mamba-3 regressors scored on the earlier *full-eval* protocol (v5–v18; not directly comparable to the other bars); solid red = the later regressors on *minival* (v19–v30); teal = the training-free SEA-RAFT+DA3 baseline; purple = the first learned refiners on top of it (v32, v33). Blue/orange/green/grey are external references. The message of the prehistory: every direct end-to-end regressor (red) plateaus *below* even the training-free SEA-RAFT+DA3 baseline (teal, mean 5.2%), and the $\Delta \mathbf{u}$ refiner v32 still loses to it (3.4%) — only when the 2D track is frozen and just the depth is refined (v33 onward, §1.7) does a learned module finally help.

1.3 v1–v32 architecture: within-frame + across-frame Vision-Mamba-3 (early tracker)

(The VSSD self-attention and cross-SSD operators used here are derived in the companion note `mamba3.attention`; this subsection records the early v1–v32 tracker built on them.)

A video clip is a sequence of F RGB frames $\mathbf{I}^{(1)}, \dots, \mathbf{I}^{(F)} \in \mathbb{R}^{H \times W \times 3}$. The point-tracking task is to maintain a set of N persistent *tracks*; for every track $n \in \{1, \dots, N\}$ and every frame $t \in \{1, \dots, F\}$ we predict

- a 3D position $\mathbf{p}_n^{(t)} \in \mathbb{R}^3$ in a shared world frame,
- a visibility / alive flag $\sigma_n^{(t)} \in [0, 1]$ — $\sigma_n^{(t)} \rightarrow 1$ when track n is observed in frame t , $\sigma_n^{(t)} \rightarrow 0$ when it has disappeared (occlusion, out-of-frame, end-of-life), and

- a spawn-confidence $\rho_n^{(t)} \in [0, 1]$ — $\rho_n^{(t)} \rightarrow 1$ on the frame at which a new track is born.

Tracks must keep identity across frames so that $\mathbf{p}_n^{(t)}$ and $\mathbf{p}_n^{(t')}$ are positions of *the same physical 3D point* at times t and t' .

This chapter assembles two Mamba-3 attention blocks of different types: **non-causal VSSD self-attention within each frame** (non-causal VSSD; companion note `mamba3_attention`), because a single image has no causal direction, and **causal SSD cross-attention across frames** (the cross-SSD operator (companion note)), because a track at time t may depend on its history at times $\leq t$ but not on the future. Together they realize a streaming tracker that is linear in patch count and linear in frame count.

1.3.1 Architecture overview

Block	Operator	Inputs	Output
Per-frame en- coder	VSSD self-attn (VSSD output; com- panion note)	frame patches $\mathbf{X}^{(t)} \in \mathbb{R}^{T \times D}$	frame features $\mathbf{Y}^{(t)} \in \mathbb{R}^{T \times D}$
Track propaga- tion	causal cross-SSD (cross-SSD output; companion note)	queries $\mathbf{Q}^{(t-1)}$ + features $\mathbf{Y}^{(t)}$	updated queries $\mathbf{Q}^{(t)}$
Position head	MLP	$\mathbf{Q}^{(t)}$	$\mathbf{p}^{(t)}, \sigma^{(t)}, \rho^{(t)}$

A *query-conditioned bank* $\mathbf{Q}^{(t)} = (\mathbf{q}_1^{(t)}, \dots, \mathbf{q}_N^{(t)})^\top \in \mathbb{R}^{N \times D}$ carries the identity of every track from one frame to the next. Unlike a free-floating learnable bank, each slot n is *tied to a specific physical point* at construction time: given the user-supplied query (x_n^q, y_n^q, t_n^q) — a pixel location and an anchor frame — the initial bank state is obtained by bilinear-sampling the encoder’s finest pyramid feature at that location:

$$\mathbf{q}_n^{(0)} = \text{BilinearSample}(\mathbf{Y}^{(t_n^q)}, \frac{x_n^q}{W} \cdot W_{\text{grid}}, \frac{y_n^q}{H} \cdot H_{\text{grid}}) \in \mathbb{R}^D, \quad (1)$$

where $W_{\text{grid}} \times H_{\text{grid}}$ is the finest pyramid level’s spatial extent. This eliminates the slot-assignment ambiguity that otherwise arises when free-floating slots are matched to ground-truth tracks via Hungarian assignment: there, the matching flips between batches and the gradient signal becomes incoherent. With (1) the slot identity is fixed by the input, slot n always means *the trajectory of physical point* (x_n^q, y_n^q, t_n^q) , and gradients accumulate coherently per slot.

At each frame t :

1. Encode the frame: $\mathbf{X}^{(t)} \rightarrow \mathbf{Y}^{(t)}$ via the VSSD operator (§1.3.2).
2. Update the bank: $\mathbf{Q}^{(t-1)} \xrightarrow{\text{cross-SSD over time}} \mathbf{Q}^{(t)}$ using $\mathbf{Y}^{(1)}, \dots, \mathbf{Y}^{(t)}$ as keys / values, with a causal mask that hides future frames (§1.3.3).
3. Read out 3D position, visibility, and spawn-confidence per track via small MLP heads (§1.3.4).

1.3.2 Within-frame block: VSSD self-attention

Each frame t goes through the same VSSD self-attention block as the companion note, independently of the others. Concretely with $H \times W = 504 \times 504$, patch $P = 14$, $D = 384$, $N = 64$:

$$\mathbf{X}^{(t)} \in \mathbb{R}^{T \times D} = \mathbb{R}^{1296 \times 384} \quad (\text{frame-}t \text{ patch tokens}), \quad (2)$$

$$\mathbf{B}^{(t)}, \mathbf{C}^{(t)} \in \mathbb{R}^{T \times N}, \quad \mathbf{V}^{(t)} \in \mathbb{R}^{T \times D}, \quad \mathbf{m}^{(t)} \in \mathbb{R}_{>0}^T, \quad (3)$$

$$\mathbf{H}^{(t)} = (\mathbf{B}^{(t)} \odot \mathbf{m}^{(t)})^\top \mathbf{V}^{(t)} \in \mathbb{R}^{N \times D}, \quad (4)$$

$$\mathbf{Y}^{(t)} = \mathbf{C}^{(t)} \mathbf{H}^{(t)} \in \mathbb{R}^{T \times D} = \mathbb{R}^{1296 \times 384}. \quad (5)$$

The choice of VSSD (non-causal) over bidirectional SSD has two motivations specific to tracking: (i) within a single frame there is no preferred direction in the patch grid, and (ii) the $N \cdot D = 24576$ -scalar global summary $\mathbf{H}^{(t)}$ is independent of T , so high-resolution tracking (large T) does not blow the within-frame budget. Compute per frame is $O(TND)$, memory is $O(ND)$ for the summary.

1.3.3 Across-frame block: causal cross-SSD

The query bank reads from the frame features via the cross-SSD operator of the cross-SSD operator (companion note), but with two crucial choices:

- (a) **Queries are the persistent bank.** Take $\mathbf{C}^{q,(t)} \in \mathbb{R}^{N \times N_{state}}$ from $\mathbf{Q}^{(t-1)}$, the previous frame’s bank — these are the *tracks asking “where am I now?”* The kv side comes from the new frame’s features $\mathbf{Y}^{(t)} \in \mathbb{R}^{T \times D}$, projected to $\mathbf{B}^{kv,(t)} \in \mathbb{R}^{T \times N_{state}}$ and used as the value matrix in place of $\mathbf{V}^{kv}$. Reading from T patches into N tracks costs $O(NTN_{state})$ in Variant B or $O(TN_{state}D)$ for the state of Variant A.
- (b) **The mask is causal in time, not in patches.** Stack the kv side over the frames seen so far, $\mathbf{Y}^{(1)}, \dots, \mathbf{Y}^{(t)} \in \mathbb{R}^{T \times D}$, and let the cross-mask $\mathbf{L}^{cross} \in \mathbb{R}^{N \times tT}$ block the future:

$$\mathbf{L}_{n,(s,j)}^{cross} = \mathbb{I}[s \leq t] \cdot \left(\prod_{k=s+1}^t \alpha_k \right) \gamma_s, \quad (6)$$

where the second factor is the same temporal decay profile as the SSD mask of (companion note), but now the running index is over frames rather than tokens, and the cell $\mathbf{L}_{n,(s,j)}^{cross}$ is shared across all patches j of frame s . The indicator $\mathbb{I}[s \leq t]$ enforces temporal causality so a track at time t cannot peek at frames $> t$. This is exactly the recipe of the cross-SSD operator (companion note) adapted from token causality to frame causality.

Concretely, the per-frame update of the bank is

$$\boxed{\mathbf{Q}^{(t)} = (\mathbf{L}^{cross} \odot \mathbf{C}^{q,(t)} \mathbf{B}^{kv,(1:t)\top}) \mathbf{V}^{kv,(1:t)} \in \mathbb{R}^{N \times D},} \quad (7)$$

where the $(1:t)$ superscript denotes vertical stacking over the first t frames. With the Mamba-3 recurrent form (streaming recurrence; companion note) the right-hand side is computed *incrementally*: maintain a frame-level hidden state $\mathbf{h}_{track}^{(t)} \in \mathbb{R}^{N \times N_{state} \times D}$ that is updated in place each frame, so the streaming cost is $O(NN_{state}D)$ per frame — independent of clip length.

1.3.4 Per-track read-out heads

Each updated query $\mathbf{q}_n^{(t)} \in \mathbb{R}^D$ passes through three small MLPs:

$$\Delta \mathbf{p}_n^{(t)} = \text{MLP}_{xyz}(\mathbf{q}_n^{(t)}) \in \mathbb{R}^3, \quad (\text{motion since query anchor}) \quad (8)$$

$$\sigma_n^{(t)} = \sigma(\text{MLP}_{vis}(\mathbf{q}_n^{(t)})) \in [0, 1], \quad (\text{visibility / alive}) \quad (9)$$

$$\rho_n^{(t)} = \sigma(\text{MLP}_{spawn}(\mathbf{q}_n^{(t)})) \in [0, 1], \quad (\text{spawn flag}) \quad (10)$$

where σ is the sigmoid. The position head predicts $\Delta \mathbf{p}_n^{(t)}$ — the motion of track n at frame t relative to its own query-anchor position — rather than absolute coordinates. A monocular model cannot recover the absolute metric scale of the scene from RGB alone, so any absolute-position prediction is identifiable only up to an unknown per-clip scale factor; predicting the relative trajectory side-steps that ambiguity entirely (the relative motion is well-defined given the camera and the trajectory shape). At inference, the absolute prediction is recovered as

$\mathbf{p}_n^{(t)} = \mathbf{p}_n^q + \Delta\mathbf{p}_n^{(t)}$ using the query anchor position $\mathbf{p}_n^q$ (which the TAPVid-3D evaluator provides as the per-track GT position at the query frame).

Stacking over all N tracks gives the predicted trajectory tensor

$$(\Delta\mathbf{p}^{(t)}, \boldsymbol{\sigma}^{(t)}, \boldsymbol{\rho}^{(t)}) \in \mathbb{R}^{N \times 3} \times [0, 1]^N \times [0, 1]^N. \quad (11)$$

1.3.5 Spawn and disappear

With query-conditioned initialisation (1), N equals the number of user-supplied queries — there is no fixed worst- case slot count to over-allocate, and no need for the Mask2Former-style “no-object” class or Hungarian-spawn matching.

Disappear. A track is treated as occluded at frame t when $\sigma_n^{(t)} < \tau_{\text{vis}}$ (e.g. $\tau_{\text{vis}} = 0.3$). Its query embedding $\mathbf{q}_n^{(t)}$ still updates from the cross-attention, so the track can *re-emerge* if it becomes visible again — the visibility flag is decoupled from the query state.

Spawn / birth frame. The query (x_n^q, y_n^q, t_n^q) specifies when track n is *seeded*; the spawn-confidence head $\rho_n^{(t)}$ predicts when within the clip the underlying physical point first becomes visible, which the loss $\mathcal{L}_{\text{spawn}}$ supervises against $\mathbb{I}[t = t_n^{\text{birth}}]$. New physical points entering the scene after the clip’s first frame are accommodated by adding a new query at the appropriate (x, y, t) ; the bank then grows by one slot without retraining.

1.3.6 Loss

Because (1) pins slot n to the query (x_n^q, y_n^q, t_n^q) , there is no slot-assignment freedom — slot n is GT track n by construction. No Hungarian matching is needed. Let $\mathbf{p}_n^{*,(t)}, \sigma_n^{*,(t)}$ be the ground-truth 3D position and visibility for that track at frame t , and let $\mathbf{p}_n^q := \mathbf{p}_n^{*,(t_n^q)}$ be the GT 3D position at the query anchor. The ground-truth motion is

$$\Delta\mathbf{p}_n^{*,(t)} := \mathbf{p}_n^{*,(t)} - \mathbf{p}_n^q, \quad (12)$$

and the model’s prediction $\Delta\mathbf{p}_n^{(t)}$ from §1.3.4 is compared against this.

Per-clip scale normalisation. Different TAPVid-3D subsets span very different metric scales (pstudio / adt are 1–5 m indoor scenes; drivetrack is 10–50 m outdoor driving). An un-normalised L1 on $\Delta\mathbf{p}$ would make a drivetrack batch’s gradient $\sim 10\times$ larger than a pstudio batch’s, swamping the optimisation. Following the TAPVid-3D evaluator’s `scaling=‘median’` convention, normalise both sides by a per-clip scale

$$s = \text{median}(\|\mathbf{p}_n^q\|_2 : n \in \mathcal{N}) + \varepsilon, \quad (13)$$

where $\mathcal{N}$ ranges over the GT tracks in the clip and ε is a small positive constant (e.g. 10^{-3} m). The scaled position residual is then

$$\mathbf{r}_n^{(t)} := \frac{1}{s} (\Delta\mathbf{p}_n^{(t)} - \Delta\mathbf{p}_n^{*,(t)}). \quad (14)$$

Loss terms (v6 — Smooth-L1, magnitude, direction-cosine, 2-D reprojection). A plain ℓ_1 on the scaled residual gives the optimiser the same constant gradient $\pm 1/s$ whether the residual is 1 cm or 1 m. With many TAPVid-3D queries moving only a few cm per 8-frame window, “predict $\Delta\mathbf{p} \approx 0$ ” is a competitive local minimum under ℓ_1 , and the model collapses there. v6 adds three terms that attack the zero-motion baseline and one that grounds the prediction in pixel space:

- **Scale-normalised Smooth- ℓ_1 position** (Huber with $\delta=1$) on visible frames:

$$\mathcal{L}_{\text{pos}} = \frac{1}{Z_{\text{pos}}} \sum_{n,t} \sigma_n^{*,(t)} \sum_{c \in \{x,y,z\}} \text{SmoothL1}(\mathbf{r}_n^{(t)}[c]), \quad (15)$$

with $Z_{\text{pos}} = \sum_{n,t} \sigma_n^{*,(t)}$. Quadratic near zero, linear far away $\Rightarrow$ small residuals receive small gradient, so the optimiser stops wasting capacity reducing already-small errors and focuses on the big-error tracks where “predict 0” is most wrong.

- **Scaled magnitude penalty:**

$$\mathcal{L}_{\text{mag}} = \frac{1}{Z_{\text{pos}}} \sum_{n,t} \sigma_n^{*,(t)} \left(\frac{\|\Delta \mathbf{p}_n^{(t)}\| - \|\Delta \mathbf{p}_n^{*,(t)}\|}{s} \right)^2. \quad (16)$$

Forces the predicted motion to have the right *magnitude ratio to scene scale*, which under a monocular camera is the only magnitude information geometrically recoverable.

- **Direction cosine:**

$$\mathcal{L}_{\text{dir}} = \frac{1}{Z_{\text{pos}}} \sum_{n,t} \sigma_n^{*,(t)} (1 - \cos(\Delta \mathbf{p}_n^{(t)}, \Delta \mathbf{p}_n^{*,(t)})). \quad (17)$$

Penalises wrong-direction motion regardless of magnitude. A model that gets the direction right but under-shoots magnitude is preferred over one that predicts the right magnitude in the wrong direction.

- **2-D pixel-space reprojection.** The TAPVid-3D queries are natively 2-D pixel locations; supervising in pixel space gives the model a geometric signal that is in-distribution with the input image and that has *no* scale ambiguity:

$$\begin{aligned} \hat{\mathbf{p}}_n^{(t)} &= \mathbf{p}_n^q + \Delta \mathbf{p}_n^{(t)}, \quad \hat{\mathbf{u}} \hat{\mathbf{v}}_n^{(t)} = \pi(\hat{\mathbf{p}}_n^{(t)}, \mathbf{K}), \\ \mathcal{L}_{\text{reproj}} &= \frac{1}{Z_{\text{pos}} W_{\text{img}}} \sum_{n,t} \sigma_n^{*,(t)} \sum_{c \in \{u,v\}} \text{SmoothL1}(\hat{\mathbf{u}} \hat{\mathbf{v}}_n^{(t)}[c] - \mathbf{u} \mathbf{v}_n^{*,(t)}[c]), \end{aligned} \quad (18)$$

where $\pi(\cdot, \mathbf{K})$ is pinhole projection through the per-clip intrinsics and W_{img} is the image side length used for training (normalises the pixel residual to $\sim [0, 1]$).

- **Visibility BCE, spawn BCE, and temporal smoothness** unchanged from § v5.

The total loss is

$$\mathcal{L} = \lambda_{\text{pos}} \mathcal{L}_{\text{pos}} + \lambda_{\text{mag}} \mathcal{L}_{\text{mag}} + \lambda_{\text{dir}} \mathcal{L}_{\text{dir}} + \lambda_{\text{reproj}} \mathcal{L}_{\text{reproj}} + \lambda_{\text{vis}} \mathcal{L}_{\text{vis}} + \lambda_{\text{spawn}} \mathcal{L}_{\text{spawn}} + \lambda_{\text{smooth}} \mathcal{L}_{\text{smooth}}, \quad (19)$$

with default weights (1, 0.5, 0.5, 1, 0.5, 0.5, 0.1).

The cross-attention’s causal mask (6) ensures that the loss at frame t depends only on frames $\leq t$; gradients flow back through the recurrent hidden state $\mathbf{h}_{\text{track}}^{(t)}$, so the network learns a streaming update rule, not a teacher-forced batch tracker.

1.3.7 Why this operator pairing

VSSD self-attention and causal cross-SSD have complementary strengths matched to the two axes of the tracking problem:

- Within a frame, structure is 2-D and orderless — every patch can in principle inform every other. VSSD reaches that all-to-all coverage at $O(T)$ cost via the rank- N global summary $\mathbf{H}^{(t)}$, instead of the $O(T^2)$ that softmax MSA or bidirectional SSD would pay (the companion note’s ratio: $\sim 20\times$ cheaper on a 504^2 image).

- Across frames, structure is 1-D and causally ordered — frame t depends on its history. Causal SSD with the temporal mask (6) preserves that ordering and admits a $O(1)$ -per-frame streaming recurrence (streaming recurrence; companion note), so the tracker’s memory does not grow with clip length. The full $\mathbf{L}^{\text{cross}}$ never needs to be materialised at inference.

The persistent query bank $\mathbf{Q}^{(t)}$ is the bridge: it is updated by both blocks (VSSD-encoded features feed it through cross-SSD), but its size $N \times D$ never depends on either the patch count T or the clip length F . The end-to-end forward pass is $O(F \cdot TND)$ in time, $O(NT_{\max}N_{\text{state}}D)$ in memory if the within-frame summaries $\mathbf{H}^{(t)}$ are kept (the default), or $O(NN_{\text{state}}D)$ if only the rolling track hidden state is kept (streaming inference).

1.4 Overview

Given a video $\{I_t\}_{t=1}^F$ and query points $\{(\mathbf{u}_n, a_n)\}_{n=1}^N$ (pixel position $\mathbf{u}_n$ at anchor frame a_n), the goal is a 3D track $\hat{\mathbf{X}} \in \mathbb{R}^{F \times N \times 3}$ and visibility $\hat{v} \in \{0, 1\}^{F \times N}$. Our pipeline (Fig. 2) has a *training-free* baseline — SEA-RAFT flow-chaining for the 2D track plus DA3-l unprojection for 3D — and a small *learned* module, v33, that refines depth.

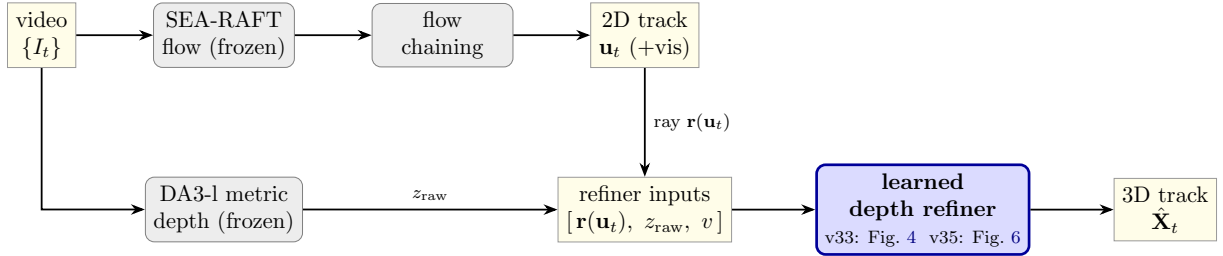


Figure 2: Overall pipeline. Grey = frozen / training-free; the **large blue box** is the only learned module (0.40M params); its internals are Fig. 4 (v33) and Fig. 6 (v35). SEA-RAFT’s 2D track $\mathbf{u}_t$ (and visibility) are *never* modified. The refiner’s *inputs* (yellow box) — the frozen pixel ray $\mathbf{r}(\mathbf{u}_t)$ (from $\mathbf{u}_t$ and the intrinsics), DA3-l’s raw depth z_{raw} , and visibility v — are exactly the yellow input boxes that sit *outside* the blue box in Fig. 4 and Fig. 6 (v35 additionally feeds a 5×5 depth patch and a DINOv3 feature). The box corrects depth along the frozen ray and unprojects to 3D, so the 2D reprojection is invariant by construction (Fig. 3).

1.5 2D track: SEA-RAFT flow-chaining (baseline, frozen)

We precompute dense forward/backward flow for consecutive frames with a frozen SEA-RAFT model, $\mathbf{f}_t^{\rightarrow}, \mathbf{f}_t^{\leftarrow}$. Each query is propagated outward from its anchor by bilinearly sampling the flow at the current position,

$$\mathbf{u}_{t+1} = \mathbf{u}_t + \mathcal{S}(\mathbf{f}_t^{\rightarrow}, \mathbf{u}_t), \quad \mathbf{u}_{t-1} = \mathbf{u}_t + \mathcal{S}(\mathbf{f}_{t-1}^{\leftarrow}, \mathbf{u}_t), \quad (20)$$

where $\mathcal{S}$ is bilinear sampling. Visibility uses a forward–backward consistency test: a hop is consistent iff $\|\mathbf{d}^{\rightarrow} + \mathbf{d}^{\leftarrow}\| \leq \alpha(\|\mathbf{d}^{\rightarrow}\| + \|\mathbf{d}^{\leftarrow}\|) + \beta$ ($\alpha=0.05$, $\beta=1$ px), and a point stays occluded once it fails. This 2D track is strong on its own and is held *fixed* for everything downstream.

1.6 Metric depth and differentiable unprojection

We use two Depth-Anything-3 (DA3) metric-depth models: **DA3-l** (DA3Metric-Large, 0.35 B) for all results through v39, and **DA3-g** (DA3Nested-Giant-Large, 1.4 B) from v40 onward (§1.12); a bare “DA3” refers to DA3-l. DA3-l supplies a per-frame metric depth map D_t (absolute

metres). We sample it at the tracked pixel and unproject with the pinhole intrinsics $K = (f_x, f_y, c_x, c_y)$:

$$z_t = \mathcal{S}(D_t, \mathbf{u}_t), \quad \hat{\mathbf{X}}_t = \left(\frac{u_t - c_x}{f_x} z_t, \frac{v_t - c_y}{f_y} z_t, z_t \right). \quad (21)$$

Sampling is done with `grid_sample` so gradients flow to the (refined) depth; the depth model itself is frozen. The baseline *SEA-RAFT+DA3-l* tracker is exactly Eq. (21) with z_t taken directly from DA3-l.

1.7 v33: Mamba-3 depth-along-ray refiner (the method)

DA3-l metric depth is accurate in *relative* structure but carries a global scale bias (e.g. $\sim 40\%$ on far-field driving scenes). v33 learns a small temporal correction to the depth, leaving the 2D track untouched. The key design choice: with the pixel $\mathbf{u}_t$ frozen, the only 3D degree of freedom that preserves the 2D reprojection is the depth z_t along the pixel ray (Fig. 3).

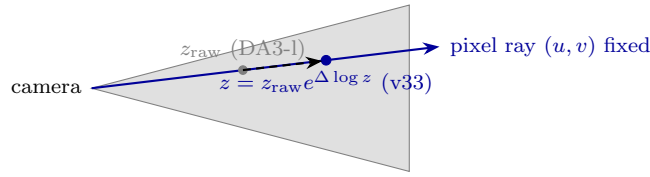


Figure 3: Depth-along-ray refinement. The tracked pixel (hence the ray direction) is frozen, so *only the depth slides along the ray* and the 2D projection is unchanged: v33 moves the point’s depth alone, leaving the predicted 2D track and visibility invariant.

Inputs and architecture. For each track n and frame t the SSM receives a 4-D feature: the normalised ray direction $(\frac{u-c_x}{f_x}, \frac{v-c_y}{f_y})$, the depth ratio $z_{\text{raw}}/z_{\text{ref}}$ (z_{ref} = per-clip median depth), and the visibility flag. The SSM *does not see* z_{raw} directly; it only sees the scale-normalised ratio. This is a deliberate training-stability choice: DA3-l depths span ~ 1 m (*adt*) to ~ 20 m (*drivetrack*), and an unnormalised input would require the same network weights to operate across a $20\times$ dynamic range.

The SSM’s role is therefore to predict a *relative* correction factor, not an absolute depth. z_{raw} bypasses the SSM entirely and is multiplied back in outside the learned module (Fig. 4):

$$\Delta \log z_t = \text{clamp}(\text{head}(h_t), \pm 2), \quad z_t = z_{\text{raw},t} \cdot e^{\Delta \log z_t}. \quad (22)$$

Because the SSM never sees the absolute scale, it cannot reason about it directly; any global-scale correction it learns must be inferred indirectly from temporal and structural patterns in the normalised input (e.g. the characteristic depth-change profile of a driving scene vs. an indoor one). A natural future improvement is to also feed $\log z_{\text{ref}}$ as a scalar context token, giving the SSM direct access to the clip-level scale without sacrificing the per-point normalisation.

Zero-initialising the head means v33 starts exactly at the SEA-RAFT+DA3-l baseline and only learns improvements. The model has ≈ 0.40 M trainable parameters.

1.8 Why depth-only (and not 2D)

An earlier variant (v32) added a learned 2D residual $\Delta \mathbf{u}$ on top of SEA-RAFT. It *degraded* accuracy: SEA-RAFT’s 2D track is already near-optimal, and small nudges push the sampling location across DA3-l depth discontinuities (object boundaries), flipping points out of the correctness band. v33 therefore freezes the 2D track entirely and refines depth only — the lever that actually carries the error.

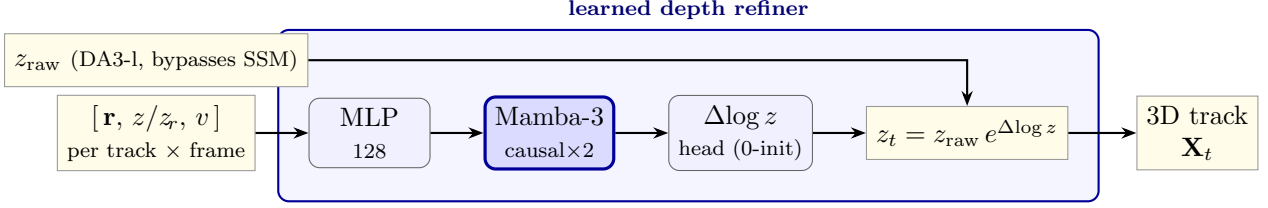


Figure 4: v33 architecture — the inside of the blue box in Fig. 2. Input abbreviations: $\mathbf{r}=(\text{ray}_x, \text{ray}_y)$ is the frozen pixel ray from the 2D track $\mathbf{u}_t$, $z/z_r=z_{\text{raw}}/z_{\text{ref}}$, v =visibility. **Bypass:** z_{raw} (DA3-l metric depth) skips the SSM entirely and is multiplied back in at the end — the SSM is a *relative corrector* that predicts the factor $e^{\Delta \log z}$, not an absolute depth. **causal $\times 2$:** two causal Mamba-3 layers (past-only; enables streaming). **4 heads:** Mamba-3 splits the 64-dim state into 4 independent heads (16-dim each), analogous to multi-head attention but computed as a linear recurrence.

1.9 v35: image-conditioned joint 2D–depth refiner

v35 revisits the joint 2D–depth correction that failed in v32, but removes its root cause. v32 nudged the pixel blindly; the SSM had no way to tell whether it was pushing toward or away from a depth discontinuity. v35 gives it that context through two new per-track inputs:

- **DINOv3 appearance features** (64-D): sampled from a frozen ViT-S/16 *DINOv3* [5] backbone (`dinov3-vits16-pretrain-lvd1689m`) at the current track position. These provide a local visual “fingerprint” of the tracked object, allowing the SSM to detect when the 2D position has drifted off the target. We use DINOv3 rather than DINOv2 specifically for its *dense* (patch-level) features. DINOv3 is Meta’s 2025 self-supervised ViT, trained on $\sim 1.7\text{B}$ images (LVD-1689M, $\sim 12\times$ DINOv2’s 142M) with a *Gram-anchoring* loss that keeps the patch-token feature map sharp and spatially consistent throughout the long training schedule — DINOv2’s dense features, by contrast, degrade as pre-training progresses. Since we sample a single feature vector at each track pixel, this patch-level fidelity is exactly what matters, so DINOv3’s dense maps are a better fit than DINOv2’s here. (The encoder class in our code is still named `DINOv2Encoder` for legacy reasons, but it loads the DINOv3 weights above.)
- **Local depth patch** (25 values). We read the DA3-l depth at a 5×5 grid of points centred on the tracked pixel $\mathbf{u}_0$: 25 points in total, each one bilinearly-interpolated depth value. Along each axis the five points sit at offsets $\{-2, -1, 0, 1, 2\} \times (\text{image.size}/14)$ pixels from $\mathbf{u}_0$, so the grid spacing and extent scale with the input size. *Concretely, for our 896-px inputs* the spacing is $896/14 = 64\text{px}$: the five offsets are $\{-128, -64, 0, +64, +128\}\text{px}$, so the grid spans a $256\times 256\text{-px}$ window around the point. We then divide all 25 depths by the depth at the *centre* point, giving 25 scale-invariant depth *ratios* (the centre ratio is exactly 1), and concatenate all 25 into the SSM input. Passing ratios rather than raw depths lets the SSM read local depth *structure* — the relative jumps that mark object boundaries — independent of how far away the point is; this is the depth-discontinuity cue that v32 lacked and overshoot without.

A second output head (also zero-initialised) predicts a 2D correction $\Delta \mathbf{u}$. Its raw output is passed through a $\tanh$ (which limits it to $[-1, 1]$) and then scaled by 2, i.e. $\Delta \mathbf{u} = 2 \tanh(\cdot)$, so the pixel can shift by at most $\pm 2\text{px}$. Crucially, after $\Delta \mathbf{u}$ shifts the pixel, depth is *re-sampled* at the corrected position from the DA3-l depth map before $\Delta \log z$ is applied. This ensures the depth and 2D corrections remain geometrically consistent (Fig. 5).

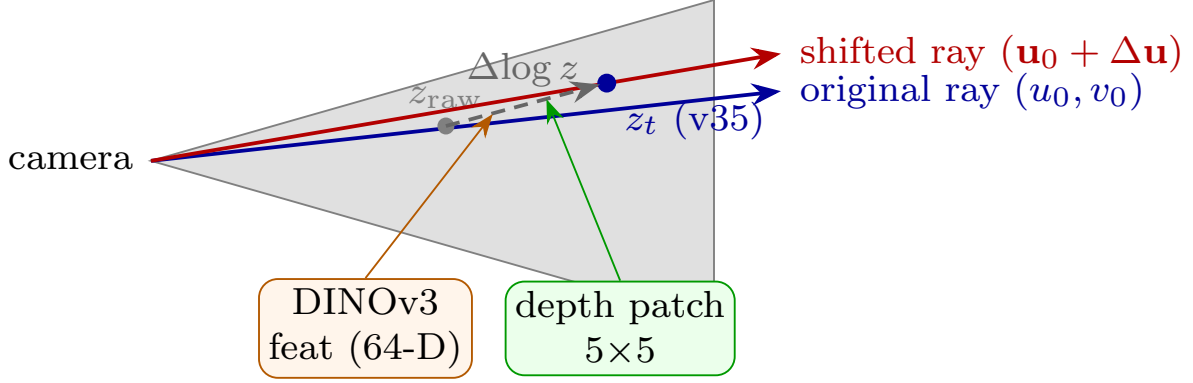


Figure 5: v35 joint correction. Two new inputs — the DINOv3 appearance feature (orange) and a local 5×5 depth patch (green) — give the model the context to predict a 2D shift $\Delta \mathbf{u}$ safely. Unlike v32, which nudged the pixel blindly and crossed depth discontinuities, v35’s 2D correction is conditioned on local appearance and local depth structure, so the informed $\Delta \mathbf{u}$ avoids depth-discontinuity artefacts. Depth is then re-sampled at the shifted pixel $\mathbf{u}_0 + \Delta \mathbf{u}$ before $e^{\Delta \log z}$ rescales it.

How the refiner uses multiple frames (the temporal axis). The figures above show what the SSM sees at a *single* frame; this paragraph makes explicit what it does *across* frames, which is the whole reason for using a state-space model instead of a per-frame MLP. **The sequence axis of the Vision-Mamba-3 SSM is time.** For one tracked point n , the refiner assembles its per-frame feature vectors into an ordered sequence $\mathbf{x}_{n,1}, \mathbf{x}_{n,2}, \dots, \mathbf{x}_{n,F}$ — one vector per video frame, each holding that frame’s ray, depth-ratio, visibility, DINOv3 feature and 5×5 depth patch — and the SSM runs *along this sequence*, carrying a hidden state $h_{n,t}$ that integrates the point’s history. The correction emitted at frame t , $z_t = z_{\text{raw},t} e^{\text{head}(h_{n,t})}$, is therefore a function of frames $1..t$ of that point, not of frame t in isolation. Concretely, in `depth_refined_tracker.py` the batch is reshaped to $(B \cdot N, F, D)$ just before the SSM, so the frame axis F is the recurrence length and each track is an *independent* sequence.

Two properties bound what “multiple frames” buys here:

- **Temporal, not spatial.** Since each of the $B \cdot N$ tracks is its own sequence, points *never exchange information*: the refiner mixes frames *within* a track but never mixes tracks *within* a frame. It uses many frames of *one point*, not many points of one frame. (Adding the missing within-frame axis is exactly the v43 proposal, §1.18.)
- **Causal; window-8 in training, whole-clip at inference.** The two Vision-Mamba-3 layers are causal (`bidirectional_mask=False`): $h_{n,t}$ sees only frames $\leq t$, keeping the model streamable. Training feeds an 8-frame window ($F=8$); at inference the *entire clip* is fed as one sequence, so the state integrates the full trajectory.

Why this helps. A single frame’s DA3 depth is noisy and globally scale-biased, and a lone MLP would have to correct each frame blind. The temporal state instead lets the refiner read how z_{raw} *evolves* along the track and (i) suppress per-frame depth flicker (a static surface should not jitter in depth), and (ii) infer the clip’s systematic scale factor from the characteristic depth-change profile of the trajectory (a driving scene’s monotone recession vs. an indoor scene’s small oscillations) — which, since the SSM never sees absolute scale (§1.7), can *only* be recovered from this temporal pattern. The frozen ray guarantees the multi-frame reasoning only ever slides depth along each point’s fixed line of sight, so it stays geometrically consistent frame-to-frame.

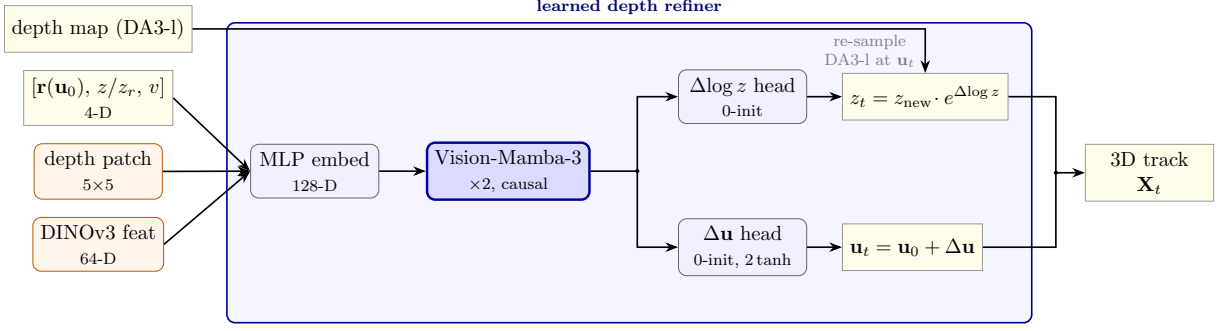


Figure 6: v35 architecture (extends Fig. 4) — again the inside of the blue box in Fig. 2. **Orange** nodes are new relative to v33. $\mathbf{u}_0$ is the 2-D pixel position from SEA-RAFT optical flow (Fig. 7); it determines the ray direction $\mathbf{r}(\mathbf{u}_0)$, the centre of the 5×5 depth patch, and the location at which the DINOv3 feature is sampled. All three inputs are concatenated, embedded by the MLP, and then fed through the Vision-Mamba-3 layers (the same causal stack as v33’s Mamba-3, but now conditioned on DINOv3 appearance — which is exactly what makes it *Vision*-Mamba-3 rather than the geometry-only Mamba-3 of v33). A second head produces $\Delta \mathbf{u} = 2 \tanh(\cdot)$ (bounded to ± 2 px). Depth is then re-sampled at the corrected pixel $\mathbf{u}_t = \mathbf{u}_0 + \Delta \mathbf{u}$ from the DA3-l depth map, and $\Delta \log z$ rescales it. Both heads are zero-initialised: v35 starts exactly at the v33 output.

1.10 v36: bidirectional flow

v36 uses the same v35 architecture; only the 2D front-end changes. It additionally runs SEA-RAFT on the *time-reversed* video and uses that flow to track the frames *before* each query’s anchor — the “reversed-video backward pass” (Fig. 7). *Important caveat.* A two-frame flow model’s output flow(I_a, I_b) does not depend on playback direction, so the reversed-video flow flow(I_t, I_{t-1}) is *identical* to the direct backward flow the forward tracker already uses. The reversed-video pass therefore leaves the 2D track unchanged: v36 is effectively identical to v35, and the sub-0.003 differences in Tables 13–15 are run-to-run (GPU) nondeterminism, not a real effect. The only flow-chaining scheme that genuinely differs fuses the forward and backward flow *within* each propagation step, $\mathbf{d} = \frac{1}{2}(\mathbf{d}^{\rightarrow} - \mathbf{d}^{\leftarrow})$, which we evaluate separately.

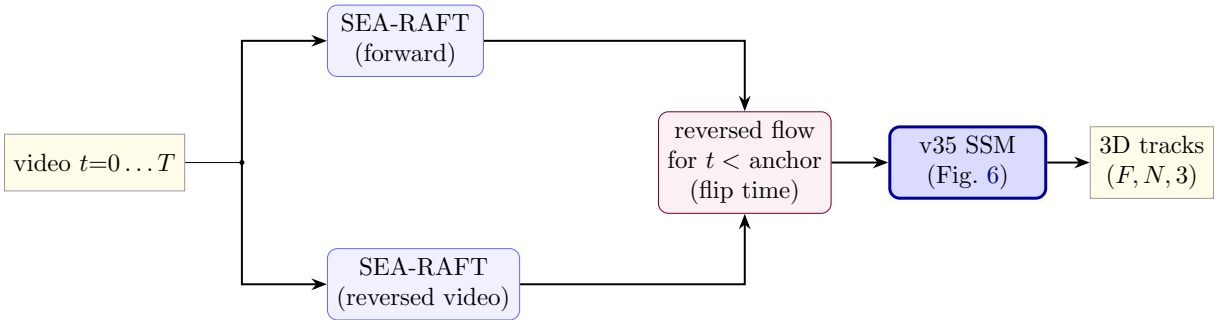


Figure 7: v36 pipeline. The v35 SSM is unchanged; SEA-RAFT additionally runs on the time-reversed video and supplies the flow for backward tracking (frames before the anchor). For a two-frame flow model this reversed-video flow is *identical* to the direct backward flow, so the reversed-video pass does not change the 2D track — v36 $\equiv$ v35 (see §1.10). It does *not* recover occluded points: an occlusion is symmetric in time, so a query anchored on one side of it cannot be tracked across the gap in either direction.

Consistent with the analysis above, v36 and v35 are effectively identical on all three subsets (Table 13, Table 15); the reversed-video pass yields no measurable gain. Evaluating the

genuinely-different per-hop fusion variant separately gives mean normalised / absolute AJ of 0.099 / 0.226 — again indistinguishable from v35 (0.096 / 0.234), confirming that no flow-chaining bidirectional scheme improves this pipeline.

1.11 v37–v39: WAFT optical-flow front-end

WAFT [2] (Warping-Alone Field Transforms, princeton-v1) is a stronger optical-flow network than SEA-RAFT. We swap it in as the 2D front-end, keeping everything downstream identical, and evaluate three variants (run at WAFT’s 512-px training resolution; scored by the same pipeline as every other row):

- **v37 (WAFT + DA3-l)** — WAFT flow + raw DA3-l unprojection, no refiner. It beats the SEA-RAFT+DA3-l baseline on both metrics (normalised $0.117 \rightarrow 0.123$, absolute $0.147 \rightarrow 0.159$): a better 2D track alone helps.
- **v38 (WAFT + DA3-l, bidir)** — the per-hop forward+backward flow fusion of §1.10. As expected it is $\approx$ v37 (0.126 / 0.159); fusion adds nothing here.
- **v39 (WAFT flow + v35 depth refiner)** — WAFT’s 2D track fed into the v35 DINOv3-SSM depth refiner. This combines the best 2D front-end with depth-scale correction and is our **best absolute-metric model**: metric-AJ 0.246, beating the SEA-RAFT-fed v35 (0.234) on all three subsets (Table 15). As with v35, the refiner trades relative-structure quality for absolute-scale correctness, so v39’s normalised AJ (0.107) sits below WAFT+DA3-l (v37, 0.123).

1.12 v40: WAFT + DA3-g metric depth (no refiner)

Pipeline: WAFT flow + DA3Nested-Giant-Large metric depth, unprojected directly (no learned refiner). A pure depth-backbone swap on the training-free baseline.

Diagnosis. Investigating the drivetrack far-field collapse (absolute-AJ ≈ 0.005 , Table 15) we measured, at the ground-truth tracked pixels, the ratio of our cached metric depth to true depth. The current backbone, DA3Metric-Large (0.35 B), systematically *under-predicts* drive-track depth by almost a factor of two, and the bias worsens with range:

median($\hat{z}/z^*$)	drivetrack	adt	pstudio
DA3Metric-Large (DA3-l, current)	0.56 (far 0.53)	0.98	0.74
DA3Nested-Giant-Large (DA3-g, 1.4 B)	1.06 (far 1.06)	0.94	0.70

So the drivetrack “collapse” is not a structure or tracking failure — it is a metre-scale bias of the small monocular metric-depth model. The larger DA3Nested-Giant-Large recovers near-perfect metric depth on drivetrack (ratio 1.06, including the far field), is roughly neutral on adt (both ≈ 1), and does not fix pstudio (both ≈ 0.7 — a separate, unsolved bias). Because the tracker consumes a precomputed depth cache, this is a drop-in backbone swap: re-cache depth with the nested model and re-run.

Result (whole minival). The swap fixes the drivetrack collapse. As a training-free baseline (no refiner), WAFT+DA3-g scores on *absolute* metric-AJ: drivetrack 0.146, adt 0.285, pstudio 0.192 (mean **0.208**) — versus the same baseline on the old DA3Metric-Large depth (v37: drivetrack 0.005, mean 0.159). The entire gain is drivetrack, where far-field error collapses from ~ 27 m to a 2.5 m median; adt/pstudio are essentially unchanged (both depth models are already near-metric there). On the scale-invariant leaderboard metric the swap barely moves the number (mean 0.093), because median rescaling removes exactly the global scale it corrects. The depth swap alone does not yet beat the current best v39 (0.246 absolute), which carries the learned refiner — closing that gap is v41 (below).

Relation to v42. This is what motivates deferring the world-frame v42 below: v42 recovers metric scale the hard way (learned pose + triangulation), whereas the far-field problem is dominated by a depth-model scale bias that the bigger depth model fixes directly. Caveats: the nested model is 1.4 B ($\sim 4\times$ slower, non-commercial license) and its metric scale is mildly resolution-sensitive.

1.13 DA3-g re-evaluation of the external SOTA baselines

The v40 result raises a fair question: is the drivetrack gain specific to our pipeline, or does the DA3-l scale bias hurt *every* method that lifts 2D tracks with DA3 depth? To answer it we re-ran each external baseline that consumes DA3 depth — SEA-RAFT+DA3, WAFT+DA3 (=v40), DELTA, SpatialTrackerV2, and TAPIP3D — swapping only the depth cache from DA3-l to DA3-g (Tables 1–2; whole-method DA3-g charts in Figs. 12–13); the 2D front-ends, models, and eval protocol are unchanged. (The DA3-l-trained *refiners* v33/v35/v39 are excluded: fed DA3-g zero-shot they over-correct, cf. §1.14. TrackCraft3R could not be re-run: its 1.3 B video-diffusion pipeline runs out of memory on the 12 GB GPU under DA3-g’s larger metric depths, on top of its 31 min/clip cost.)

Table 1: External SOTA on **DA3-l** depth (old DA3Metric-Large): absolute metric-AJ on TAPVid-3D minival (higher better), per subset.

Method + DA3-l	drivetrack	pstudio	adt	mean
SEA-RAFT	0.005	0.165	0.273	0.147
WAFT (v37)	0.005	0.181	0.289	0.159
DELTA	0.006	0.199	0.344	0.183
SpatialTrackerV2	0.008	0.192	0.179	0.126
TAPIP3D	0.006	0.131	0.164	0.100

Table 2: External SOTA on **DA3-g** depth (new DA3Nested-Giant-Large): absolute metric-AJ on TAPVid-3D minival (higher better), per subset. [†]adt on 22/50 clips ($s_{\text{wind}}=60$ OOM on the rest under the larger metric depths). [‡]adt OOM-limited; mean over drivetrack+pstudio only.

Method + DA3-g	drivetrack	pstudio	adt	mean
SEA-RAFT	0.136	0.175	0.269	0.193
WAFT (v40)	0.146	0.192	0.285	0.208
DELTA	0.164	0.194	0.324	0.227
SpatialTrackerV2	0.043	0.184	0.170 [†]	0.132
TAPIP3D	0.110	0.148	— [‡]	0.129 [‡]

The pattern is unambiguous: on *drivetrack* the DA3-l→DA3-g swap lifts absolute metric-AJ by 18–29 $\times$ for every method that unprojects depth directly (SEA-RAFT, WAFT, DELTA, TAPIP3D all jump from ≈ 0.006 to 0.11–0.16), while pstudio and adt stay within a few points. The lone exception, SpatialTrackerV2, gains little ($0.008 \rightarrow 0.043$) because it consumes depth only as a weak prior in window-local coordinates rather than unprojecting it. On the scale-invariant leaderboard metric the swap barely moves any method (median rescaling removes the global scale it fixes). Conclusion: the drivetrack “collapse” is a property of the DA3-l depth model shared across the field, not of any one tracker, and the bigger DA3-g backbone fixes it for all of them.

1.14 v41: DA3-g depth + vmamba3 refiner

v41 puts the v35 DINOv3-conditioned refiner back on top of the v40 baseline, but the refiner is **retrained on DA3-g depth**. The v39 refiner was trained on the $0.55\times$ -biased DA3Metric-Large depth and learned to scale depth up, so applied zero-shot to the now-metric DA3-g depth it *over-corrects* (drivetrack absolute-AJ collapses to 0.003, median error 18.6 m). Retraining unlearns exactly that: warm-started from the v39 refiner, at step 0 it sits at loss 1.04 (still over-correcting) and drops below 0.06 within 50 steps. We precomputed DA3-g depth for the full 4,418-clip training split and retrained the v35 refiner (0.44 M params, 20k steps), evaluated with the WAFT front-end (the v41 analogue of v39).

Table 3: v41 vs. its lineage: absolute metric-AJ on TAPVid-3D minival (higher better), per subset. Best per column in bold. v41 wins drivetrack but regresses on the near-field subsets, so v39 stays best overall.

Pipeline	drivetrack	pstudio	adt	mean
v39 (DA3-l + refiner)	0.141	0.279	0.318	0.246
v40 (DA3-g, no refiner)	0.146	0.192	0.285	0.208
v41 (DA3-g + refiner)	0.166	0.198	0.285	0.216

The result is a *targeted* win, not an overall one (Table 3). On drivetrack the metric depth pays off exactly as intended: absolute metric-AJ rises $0.141 \rightarrow 0.166$ (+18%) and mean 3D error drops $4.67 \rightarrow 3.90$ m. The refiner also earns its keep on DA3-g — v41 (0.216 mean) beats the refiner-free v40 (0.208), confirming the learned along-ray and $\Delta\mathbf{u}$ corrections still add value once the scale bias is gone. But pstudio and adt *regress* ($0.279 \rightarrow 0.198$ and $0.318 \rightarrow 0.285$), dragging the mean below v39 ($0.246 \rightarrow 0.216$); the scale-invariant leaderboard metric moves the same way (mean $0.107 \rightarrow 0.080$). So **v39 remains the best single pipeline overall**: DA3-g helps the outdoor far-field but hurts the indoor/egocentric near-field.

A per-domain depth choice (DA3-l indoors, DA3-g on driving) would beat both (≈ 0.254 mean), but we do not pursue a mixture — we chase a single method. That makes the open question sharp: *why is the 1.4 B metric-accurate DA3-g worse than the smaller, scale-biased DA3-l in the near field* (pstudio/adt), where a global scale bias should barely matter?

Diagnosis: the refiner corrects systematic bias, not noise. Isolating the refiner’s effect — no-refiner vs. refiner on the *same* depth and the same WAFT 2D front-end (v37 $\rightarrow$ v39 for DA3-l, v40 $\rightarrow$ v41 for DA3-g) — answers it (Table 4). The refiner genuinely reshapes depth per-point: on DA3-l it *halves* the pstudio depth error ($0.732 \rightarrow 0.348$ m). But it barely touches DA3-g’s ($0.777 \rightarrow 0.734$ m), even though the two *raw* depths start at nearly the same error. A learned corrector can only undo error that is *predictable* from its inputs (appearance, ray, local depth patch) — systematic bias — not unstructured noise. That is exactly what separates the two depths in the near field: after removing each model’s own per-clip scale, DA3-l’s pstudio residual is 0.073 m (tight, systematic) while DA3-g’s is 0.238 m ($3\times$ looser). Systematic bias the refiner learns away; noise survives refinement, keeping DA3-g’s metric *and* normalized scores low — consistent with metric-correct $\Rightarrow$ normalized-correct, since DA3-g’s near-field depth is simply not metric-correct, only noisier. In short, the giant nested model trades near-field shape fidelity for the far-field metric correctness that rescues drivetrack. One residual possibility — that pstudio (only 3.5% of training clips) is under-weighted rather than intrinsically un-correctable — is tested by a pstudio-weighted retrain (§1.15).

Table 4: Refiner effect on pstudio depth (median 3D error, m), same WAF 2D front-end. The refiner halves DA3-l’s error but barely moves DA3-g’s — it corrects systematic bias, not the noise DA3-g carries in the near field.

pstudio depth	raw (no refiner)	refined	Δ
DA3-l	0.732 (v37)	0.348 (v39)	−52%
DA3-g	0.777 (v40)	0.734 (v41)	−6%

1.15 v41b: pstudio-only retrain (systematic-vs-noise check)

§1.14 left one alternative open: DA3-g’s stubborn pstudio error might be intrinsic noise, or merely under-weighted in training (pstudio is only $\sim 2.4\%$ of training clips). To decide, we fine-tune the v41 refiner on *pstudio only* (all 106 pstudio training clips, DA3-g depth, warm-started from v41) — giving the refiner maximal opportunity to learn the pstudio correction — and re-evaluate on the 50 held-out pstudio minival clips (Table 5).

Table 5: Pstudio-only retrain (v41b): held-out pstudio minival depth error and absolute metric-AJ. Focusing the refiner on pstudio helps (error $0.734 \rightarrow 0.563$ m), but plateaus well above what the same refiner reaches on DA3-l (v39, 0.348 m) — a residual noise floor no weighting closes.

pstudio pipeline	err median (m)	metric-AJ
raw DA3-g (no refiner, v40)	0.777	0.192
DA3-g + standard refiner (v41)	0.734	0.198
DA3-g + refiner, pstudio $\times 20$ (v42)	0.652	0.213
DA3-g + pstudio-only refiner (v41b)	0.563	0.225
DA3-l + refiner (v39)	0.348	0.279

The answer is *both, but noise binds*. Under-weighting was real but partial: 100% pstudio focus does help (error $0.734 \rightarrow 0.563$ m, metric-AJ $0.198 \rightarrow 0.225$), so some of DA3-g’s pstudio error is systematic and the standard v41 genuinely under-served pstudio. But it *plateaus at 0.563 m and cannot reach DA3-l’s 0.348 m* — and v39 hit that from the *same* 106 clips at far less weight, so the gap is the depth, not the data budget. But §1.17 dissects that residual and corrects its attribution: it is *not* spatial depth noise (DA3-g’s per-frame pstudio shape is in fact as good as DA3-l’s) but a per-frame *scale* instability in DA3-g that the per-track refiner cannot observe or correct; DA3-l’s temporally stable (if biased) scale is what lets its refiner reach 0.348 m.

Consequence for a single method. Even an optimally pstudio-weighted single DA3-g pipeline tops out near 0.225 metric-AJ on pstudio; plugged into the per-subset numbers (drivetrack 0.166, adt 0.285) the mean is ≈ 0.225 , still *below* v39’s 0.246 — DA3-g’s drivetrack win (+0.025) cannot offset its near-field noise floor (pstudio -0.054 , adt -0.033). So **v39 (DA3-l) is the ceiling for a single method**; only the per-domain hybrid (§1.14) beats it. The DA3-g swap is a far-field lever, not a route to a single SOTA pipeline; beating v39 everywhere needs DA3-g’s per-frame scale to be as temporally stable as DA3-l’s (§1.17), which the per-frame-cached nested model is not.

1.16 v42: near-field boost by pstudio oversampling

§1.15 showed DA3-g’s pstudio error is *partly* systematic — a pstudio-only retrain lifts metric-AJ $0.198 \rightarrow 0.225$ — and projected that an optimally near-weighted *single* DA3-g pipeline would top out near 0.225 mean. v42 tests that projection with a deployable recipe: the v41 configuration (WAF + DA3-g + v35 refiner) retrained for the full 20k steps, warm-started from v41, with the

106 pstudio training clips **oversampled** $\times 20$ so the under-represented near-range domain (2.4% of clips) reaches batch-mix parity with drivetrack/adt. Unlike the pstudio-only v41b checkpoint, v42 serves all three subsets at once.

Table 6: v42 (pstudio $\times 20$ oversample) vs. its DA3-g lineage: absolute metric-AJ on TAPVid-3D minival (higher better), per subset. v42 improves v41 on *all three* subsets and reaches the ≈ 0.225 mean predicted in §1.15, yet stays below the DA3-l v39 (0.246).

Method	drivetrack	pstudio	adt	mean
v40 (DA3-g, no refiner)	0.146	0.192	0.285	0.208
v41 (DA3-g + refiner)	0.166	0.198	0.285	0.216
v42 (DA3-g + refiner, pstudio $\times 20$)	0.169	0.213	0.292	0.225
v39 (DA3-l + refiner, ceiling)	0.141	0.279	0.318	0.246

Findings.

1. **A uniform, near-free gain.** Oversampling lifts pstudio $0.198 \rightarrow 0.213$ *without* costing the far subsets — drivetrack ($0.166 \rightarrow 0.169$) and adt ($0.285 \rightarrow 0.292$) both nudge up — so v42 is the **best of our DA3-g pipelines** at mean 0.225, ahead of v41 (0.216) and refiner-free v40 (0.208). The scale-invariant leaderboard mean moves the same way, $0.080 \rightarrow 0.086$.
2. **It essentially matches the strongest external DA3-g method.** At mean 0.225, v42 ties DELTA+DA3-g (0.227, Table 16) — *ahead* on drivetrack (0.169 vs. 0.164) and pstudio (0.213 vs. 0.194), behind only on adt (0.292 vs. 0.324). The residual gap to DELTA is therefore entirely adt, DELTA’s architectural stronghold, not a near-field deficit.
3. **It lands exactly on the predicted ceiling.** v42’s 0.225 mean confirms the §1.15 projection empirically at full training length.
4. **The near-field noise floor still binds.** v42’s pstudio (0.213) sits *below* the pstudio-only v41b (0.225) — one pipeline cannot spend all its capacity on pstudio — and far below the DA3-l v39 (0.279); pstudio median error is 0.652m against DA3-l’s 0.348m. Oversampling extracts the systematic part of the error but cannot cross DA3-g’s intrinsic near-range depth noise.
5. **v39 remains the single-method ceiling.** At 0.225 the best single DA3-g pipeline still trails v39’s 0.246 by 0.021; the +0.003 drivetrack edge cannot offset the near-field deficit. Beating v39 everywhere needs either the per-domain hybrid (§1.14) or a depth backbone that is metric *and* low-noise in the near field.

1.17 DA3-l vs. DA3-g: architecture, loss, and where the near-field error really is

The depth-backbone swap (DA3-l $\rightarrow$ DA3-g, §1.12–§1.16) helped far-field drivetrack but not the near field, and the refiner could halve DA3-l’s near depth error ($0.73 \rightarrow 0.35$ m) but barely move DA3-g’s ($0.78 \rightarrow 0.73$ m). This section consolidates *why*, from two angles: the two models’ architecture and training (read from the DA3 code and paper [3]), and a direct per-point error analysis on the minival that *overturns* the natural “DA3-g depth is noisier” explanation.

DA3-l — distilled monocular depth. DA3-l (da3metric-large) is a *single monocular* network: a DINOv2 ViT-L backbone with a plain DPT depth head (one output channel). It is trained by *distillation* — a teacher trained on *synthetic* data emits dense, clean pseudo-depth —

and made metric only by a canonical-focal-length rescale (f^c/f , $f^c=300$). Distilling a smooth synthetic teacher gives it low-variance, temporally-stable *shape and scale*; the focal rescale is a *deterministic* per-frame function (no fitting), so it introduces a global scale *bias* but no temporal wobble.

DA3-g — nested multi-view geometry + metric fusion. DA3-g (da3nested-giant-large) is a *nested* model of two branches: an “any-view” multi-view geometry branch (DINOv2 ViT-g with cross-view attention and a *depth-ray* DualDPT head, two channels) *plus* the DA3-l branch itself. The any-view branch is trained *scale-invariant* (ground truth normalised by the scene point map’s mean ℓ_2 norm) with a confidence-weighted ℓ_1 depth loss alongside ray-map, point-cloud and gradient terms, on *real* multi-view reconstructions (COLMAP and active sensors) the DA3 authors themselves call “noisy or sparse.” At inference (`NestedDepthAnything3Net`, `model/da3.py`) the nested model does *not* copy DA3-l’s depth; it fits a *single global scalar* s each frame by least squares so the any-view depth best matches DA3-l, then sets DA3-g = any-view $\times s$. **So DA3-g’s per-pixel shape is entirely the any-view branch’s; the metric branch (= DA3-l) contributes only that one scalar — its own clean shape is discarded.** Two asymmetries follow: (i) DA3-g’s shape is supervised by dirtier real-reconstruction geometry, not the clean synthetic teacher, so more parameters cannot buy back cleanliness the labels lack; (ii) the single *global* scalar cannot fix any *within-clip* scale variation in the any-view branch.

Why DA3-g’s scale drifts frame to frame, though DA3-l’s is stable. A natural objection: DA3-g’s scale comes from DA3-l, and DA3-l’s scale is temporally stable — so why does DA3-g’s per-frame scale wobble $\times 4.3$ more (Table 7)? Because DA3-g does *not* inherit DA3-l’s scale; it inherits a per-frame least-squares *fit* $s = \arg \min_s \|a - sb\|^2 = \sum_i a_i b_i / \sum_i b_i^2$ (with a = DA3-l depth, b = any-view depth), and the *fit* — not DA3-l — is the noisy step. Four things make s wobble even though DA3-l is steady:

1. **Re-fit independently every frame, no temporal coupling.** DA3-l’s own scale is a deterministic focal formula (no optimisation); the nested model *adds* a per-frame fit, and any independent per-frame estimate carries variance — nothing ties frame t ’s s to frame $t-1$ ’s.
2. **It reconciles two *differently-structured* depth fields.** DA3-l (smooth monocular) and the any-view branch (multi-view geometry) do not agree pixel-for-pixel, so the single best-fit scalar depends on the depth distribution and the visible-point set — which change with scene content frame to frame. Same two stable models, different overlap $\Rightarrow$ different s .
3. **The least-squares scalar is b^2 -weighted, so the *far field* sets it.** $s = \sum a_i b_i / \sum b_i^2$ is a mean of the ratio a_i/b_i weighted by any-view depth *squared*, hence dominated by the largest-depth pixels (and the confidence/non-sky mask). In a near-field indoor scene (pstudio) the “far” is the sparse, occlusion-prone background — a noisy, content-dependent anchor that jumps as the visible far surfaces change. This is why pstudio drifts most.
4. **A multi-view model run single-frame.** Our cache feeds the any-view branch one frame at a time, so its own per-frame geometry/scale is under-constrained and more variable, injecting further noise into the fit.

So DA3-l’s stability sets a stable *target*, but the drift is manufactured *at fusion time* by a per-frame, far-pixel-weighted least-squares fit against a wobbly multi-view branch over a changing mask — the fusion never copies DA3-l’s scale, it re-solves for it each frame. This is exactly why a temporally smooth scale (the offline `lprof` re-anchor) or a per-frame-correct scale (oracle) removes the drift, and what v43’s per-frame scale head (§1.18) learns to undo.

Where the error actually is (per-point analysis). We test the “DA3-g is noisier” story directly (`scripts/analyze_da3lg_shape_crossover.py`): on all 150 minival clips we sample both depths at the ground-truth pixel, remove a scale to expose shape, and measure the scale-invariant error $|\log(s\hat{z}/z^*)|$ stratified by distance (Fig. 8). Removing the scale two ways is the crux: *per-frame* (one scale per frame) isolates spatial shape; *per-clip* (one scale for the whole clip) additionally exposes temporal scale drift (Table 7).

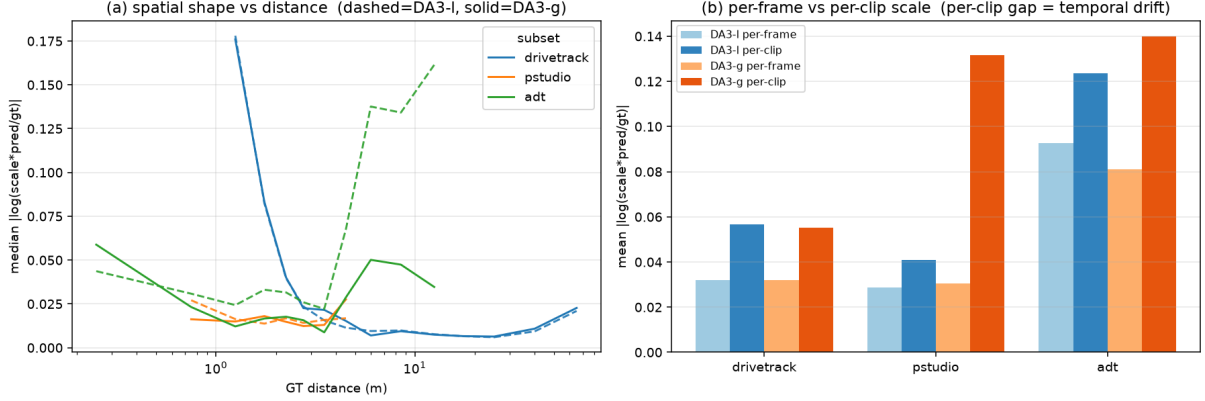


Figure 8: DA3-l vs. DA3-g depth error at GT pixels, TAPVid-3D minival. **(a)** Median scale-invariant *spatial* shape error vs. distance (per-frame scale removed): DA3-g (solid) matches or *beats* DA3-l (dashed) almost everywhere, decisively so on adt — there is no “near-field shape” deficit and no clean distance crossover, so a distance-gated depth fusion has nothing to fix. **(b)** Mean error under per-frame vs. per-clip scale alignment: DA3-l barely changes, but **DA3-g’s pstudio error explodes** $\times 4.3$ ($0.031 \rightarrow 0.132$) when only one scale per clip is allowed — i.e. its per-frame *scale* drifts over time. The near-field defect is temporal scale instability, not spatial depth noise.

Table 7: Mean scale-invariant error $|\log(s\hat{z}/z^*)|$ under per-frame vs. per-clip scale alignment. Per-frame $\approx$ per-clip $\Rightarrow$ temporally stable scale; a large jump $\Rightarrow$ the per-frame scale drifts. DA3-g’s pstudio jump ($\times 4.3$) is the near-field culprit — its *shape* (per-frame) is as good as DA3-l’s.

subset	DA3-l		DA3-g	
	per-frame	per-clip	per-frame	per-clip
drivetrack	0.032	0.057	0.032	0.055
pstudio	0.029	0.041	0.031	0.132
adt	0.093	0.124	0.081	0.140

Conclusion — and a correction. The earlier reading (§1.15) that DA3-g carries “intrinsic near-field *noise*” was measured with a per-clip scale and is wrong in attribution: DA3-g’s per-frame spatial shape on pstudio is as good as DA3-l’s (0.031 vs. 0.029); what is $4.3\times$ worse is its *per-frame scale*, which wobbles over time (its per-frame depth cache re-fits the nested model’s least-squares scalar independently each frame, whereas DA3-l’s focal rescale is deterministic and stable). Because removing one scalar *per frame* collapses the error, the defect is low-dimensional — one wrong global scale per frame, shared by all points — and hence fixable, unlike per-pixel noise. This also explains why the per-track refiner cannot repair it: it never sees absolute scale (§1.7) and, mixing only along time within a track, cannot estimate a *per-frame, all-points-shared* correction. **The lever is therefore per-frame scale stabilisation of DA3-g, not a spatial depth fusion:** either (i) re-anchor DA3-g’s per-frame scale to a temporally stable

reference (e.g. DA3-l’s smooth scale profile) offline, or (ii) add a per-frame scale head fed by *within-frame* pooling (the within-frame Vision-Mamba-3 mixing of §1.18), so one scale per frame is estimated from all points in that frame.

Offline validation. Both the diagnosis and a fix hold up without retraining (`scripts/generate_da3g_reanchor`): we rescale DA3-g’s pstudio depth by *one scalar per frame* and re-evaluate with the v42 refiner (WAFT front-end), Table 8. An *oracle* per-frame scale (from GT) more than *doubles* the scale-invariant leaderboard 3D-AJ (0.026 → 0.064) — direct proof that per-frame scale drift, not spatial shape, dominates pstudio’s leaderboard score. A *GT-free* re-anchor that gives DA3-g the temporally stable DA3-l scale profile (preserving DA3-g’s clip-average scale, so no bias is imported) improves *both* metrics with the existing refiner: normalized 0.026 → 0.043 (+62%) and absolute metric-AJ 0.213 → 0.227 (past the pstudio-only v41b, 0.225), median error 0.652 → 0.584 m. The oracle’s absolute metric-AJ instead dips (0.176) only because the refiner was trained on un-rescaled depth — retraining the refiner on re-anchored depth is the in-model next step.

Table 8: Offline per-frame re-anchoring of DA3-g depth on pstudio minival (v42 refiner, WAFT). One scalar per frame; normalized = leaderboard 3D-AJ, absolute = metric-AJ. The GT-free DA3-l-profile re-anchor improves both metrics with no retraining.

DA3-g pstudio depth	normalized 3D-AJ	absolute metric-AJ	err median (m)
baseline (v42)	0.026	0.213	0.652
+ oracle per-frame scale (GT)	0.064	0.176	0.907
+ DA3-l scale profile (GT-free)	0.043	0.227	0.584

1.18 v43: in-model per-frame scale head

§1.17 localised DA3-g’s near-field defect to a *per-frame global scale* that drifts over time; removing it offline (even by borrowing DA3-l’s scale profile) lifts pstudio on both metrics. v43 learns that correction *in-model*. Two axes of information matter — across frames (time) and across points (within a frame) — and v43 keeps them separate.

Where the multiple-frame information comes from (inherited from v35). v43 adds *no* new temporal model. It inherits v35’s: as detailed in §1.9 (“how the refiner uses multiple frames”), the Vision-Mamba-3 refiner (DINOv3-conditioned, hence *Vision*-Mamba-3, not the geometry-only Mamba-3 of v33) already consumes the whole clip along the *time* axis — for each tracked point n , its per-frame feature vectors form a sequence $\mathbf{x}_{n,1}, \dots, \mathbf{x}_{n,F}$, the batch is reshaped to $(B \cdot N, F, D)$, and its *Vision-Mamba-3 state-space model runs causally along that frame sequence, one track at a time* (no cross-track mixing). So every point already carries its own temporal history. That — and nothing else — is how v43 (and v35) uses multiple frames.

What v43 adds: the orthogonal, within-frame axis. The per-frame scale is one number *shared by all points in a frame*, so it cannot be read from any single track’s temporal history — it needs aggregation *across points within a frame*, the one axis the per-track SSM never touches. v43 adds exactly that, as a small stage after the temporal SSM:

1. Run the full v35 pipeline (§1.9): inputs → embed → per-track temporal Vision-Mamba-3 SSM → the $\Delta \log z$, $\Delta \mathbf{u}$ heads, giving per-point features $\mathbf{x} \in \mathbb{R}^{B \times F \times N \times D}$ and a first depth $z_{t,n}$.
2. **Within-frame pooling.** For each frame f *independently*, take the plain masked *mean* of the per-point features over the visible points, $\bar{\mathbf{x}}_f = (\sum_n v_{f,n} \mathbf{x}_{f,n}) / (\sum_n v_{f,n})$ with

$v_{f,n} \in \{0,1\}$ the visibility, giving one per-frame vector $\bar{\mathbf{x}}_f \in \mathbb{R}^{B \times F \times D}$. The points do *not* interact — it is a flat average, not attention.

3. **Per-frame scale head.** A zero-initialised MLP maps each $\bar{\mathbf{x}}_f$ to one bounded log-scale $\Delta s_f = s_{\max} \tanh(\text{MLP}(\bar{\mathbf{x}}_f))$, with $s_{\max}=0.5$.
4. **Apply.** Rescale *every* point in frame f by that single scalar, $z_{t,n} \leftarrow z_{t,n} e^{\Delta s_f}$, then unproject. Zero-init $\Rightarrow \Delta s_f=0 \Rightarrow$ v43 equals v42 at step 0.

Because the pooled $\bar{\mathbf{x}}_f$ come *out of* the temporal SSM, they are already temporally aware; the pooling adds the *spatial* aggregation, so the head can estimate the frame’s true global scale — the one-scalar-per-frame wobble no per-track model can see. v43 warm-starts from v42 and trains on DA3-g depth. It is a deliberately *minimal* slice of §1.21: only a per-frame global *scale*, not a per-frame pose or per-point motion.

Two shortcomings of v43, and the proposed v44. v43 recovers only part of the offline gain (Table 9), and its two structural limits say why. (i) *The pooling is a flat average, with no interaction between points.* A masked mean already drops the *invisible* points, so occlusion is *not* the issue — the issue is that among the *visible* points it cannot tell a reliable near-field anchor from a visible-but-noisy far or dynamic one (exactly the points that corrupt the least-squares scale, §1.17), and it discards the point-to-point arrangement that pins the true scale. Letting the N points *interact* first — a *bidirectional within-frame Vision-Mamba-3 self-attention* over the point axis (reshape to $(B \cdot F, N, D)$, order-agnostic, linear in N), *then* pool — turns the flat average into a learned, robust, structure-aware aggregation. The novelty over v43 is the *attention between points*, not the visibility mask (which v43 already has). (ii) *The scale is applied after the refiner, not to the depth it consumes.* v43 rescales the *output* $z_{t,n}$, so the v35 refiner still ingests the flickering DA3-g depth — its z/z_{ref} input drifts frame to frame — and must work against it. It is cleaner to *de-flicker first*: a small within-frame module (Mamba-3 / MLP over the frame’s points) estimates the per-frame scale and produces a stabilised depth $z^{\text{stab}} = z_{\text{raw}} e^{\Delta s_f}$, and the standard v35 refiner then runs on z^{stab} — so its along-ray $\Delta \log z$ and $\Delta \mathbf{u}$ corrections operate on temporally consistent depth instead of fighting the drift. This two-stage form, *de-flicker* $\rightarrow$ *v35 refine* (with the within-frame attention of (i) as the de-flicker’s aggregator), is the proposed **v44**.

Result. Trained 20k steps warm-started from v42 (masked-mean within-frame pooling), v43 improves v42 on *both* metrics across *all three* subsets, with the gain concentrated exactly where the drift lives — pstudio and adt — while drift-free drivetrack is flat (Table 9). Absolute metric-AJ rises to a mean 0.229, the **best of any single DA3-g pipeline** and now *ahead of* DELTA+DA3-g (0.227, Table 16), though still below the DA3-l v39 (0.246); the scale-invariant leaderboard mean rises $0.086 \rightarrow 0.092$. On pstudio the learned head recovers part of the offline re-anchor’s gain — normalized $0.026 \rightarrow 0.035$ (vs. lprof’s 0.043), absolute $0.213 \rightarrow 0.220$ (vs. lprof’s 0.227) — so the mechanism is right, but masked-mean pooling leaves headroom. Replacing it with the bidirectional within-frame Vision-Mamba-3 mixing, so the head sees point-to-point structure rather than a mean, is the natural next lever.

1.19 v44: DA3-g de-flicker (standalone)

v43 applied its per-frame scale correction *after* the v35 refiner, so the refiner still ingested the flickering depth (§1.18). v44 fixes the ordering and factors the de-flicker out as a *standalone* method, comparable head-to-head with the refiner-free WFT+DA3-g baseline (v40): its only job is to remove DA3-g’s per-frame scale drift (§1.17); it carries no along-ray or 2D correction.

Table 9: v43 (in-model per-frame scale head) vs. v42, TAPVid-3D minival. The scale head lifts both metrics on the drift-affected subsets (pstudio, adt) and leaves drift-free drivetrack flat. Absolute mean 0.229 is the best single DA3-g pipeline, ahead of DELTA+DA3-g (0.227), still below DA3-l v39 (0.246).

subset	normalized 3D-AJ		absolute metric-AJ	
	v42	v43	v42	v43
pstudio	0.026	0.035	0.213	0.220
drivetrack	0.118	0.119	0.169	0.171
adt	0.112	0.123	0.292	0.297
mean	0.086	0.092	0.225	0.229

Method. The correction is a single log-scale per frame, estimated along the *time* axis:

1. For each frame f , mean-pool the visible points’ geometric features $[\mathbf{r}, z_{\text{raw}}/z_{\text{ref}}, v]$ into one *frame token* $\mathbf{g}_f$.
2. A *time-axis* Mamba-3 (bidirectional — a de-flicker may look ahead) mixes the sequence $\mathbf{g}_1, \dots, \mathbf{g}_F$ across frames, modelling the scale trajectory directly.
3. A zero-initialised head emits $\Delta s_f = s_{\text{max}} \tanh(\cdot)$; every point in frame f is rescaled $z \leftarrow z_{\text{raw}} e^{\Delta s_f}$ and unprojected along the frozen ray.

Zero-init $\Rightarrow$ v44 starts *exactly* at WAFT+DA3-g (v40). It uses plain Mamba-3, not the 2D-image vmamba3 machinery of v35: the flicker is a purely temporal, per-frame-*global* phenomenon, so the model is a light time-axis sequence model over frame tokens (~ 0.3 M params, no DINOv3, no depth patch). Loss is 3D-only (2D track frozen); trained from scratch, 20k steps, DA3-g depth. v44 is the *learned* counterpart of the **lprof** offline re-anchor (§1.17) — both replace DA3-g’s per-frame scale with a temporally consistent one, but v44 learns it end-to-end from the 3D loss.

Result — the standalone de-flicker does not help. Trained 20k steps, v44 lands essentially *at* the WAFT+DA3-g (v40) baseline on the scale-invariant leaderboard metric and slightly *below* it on absolute (Table 10): mean normalized 0.093 (v40: 0.093), mean absolute 0.203 (v40: 0.208). It does *not* reproduce the offline **lprof** re-anchor’s pstudio gain (normalized 0.039 vs. lprof’s 0.043, §1.17), and it mildly *regresses* drift-free drivetrack (absolute 0.146 $\rightarrow$ 0.123). So a per-frame scale learned from mean-pooled *sparse tracked points* — under a per-clip-normalised 3D loss — is too weak a signal to recover the drift that the dense offline re-anchor removes, and without any along-ray refinement it applies scales that can hurt where there is no drift. The lesson: the de-flicker is not useful in isolation; it needs the refiner (v45).

1.20 v45: v44 de-flicker + v35 refiner

v45 is the two-stage pipeline §1.18 argued for — **de-flicker first, then refine**. Stage 1 is v44: it produces a stabilised depth $z^{\text{stab}} = z_{\text{raw}} e^{\Delta s_f}$ (and the correspondingly rescaled depth map). Stage 2 is the standard v35 refiner (§1.9), run on z^{stab} instead of raw DA3-g depth, so its along-ray $\Delta \log z$ and $\Delta \mathbf{u}$ corrections operate on temporally consistent depth. Where v43 bolts a scale onto the refiner’s *output* and v44 de-flickers with *no* refinement, v45 gives the refiner clean *input* — combining v44’s temporal stability with v35’s per-point sharpening.

Table 10: v44 (standalone DA3-g de-flicker) vs. the WAFT+DA3-g baseline (v40), TAPVid-3D minival. The learned de-flicker neither improves the leaderboard metric nor matches the offline lprof re-anchor, and slightly regresses drivetrack.

subset	normalized 3D-AJ		absolute metric-AJ	
	v40	v44	v40	v44
pstudio	0.039	0.039	0.192	0.196
drivetrack	0.118	0.118	0.146	0.123
adt	0.123	0.123	0.285	0.289
mean	0.093	0.093	0.208	0.203

Result — best DA3-g *absolute* accuracy, but v43 keeps the leaderboard. Warm-started from v44 (stage 1) + v42 (stage 2) and fine-tuned 20k steps, v45 gives the **best absolute metric-AJ of any single DA3-g pipeline**: mean 0.232 (Table 11), ahead of v43 (0.229), v42 (0.225) and DELTA+DA3-g (0.227), and best per subset on drivetrack (0.173) and pstudio (0.226, essentially matching the offline lprof re-anchor’s 0.227), tied with v43 on adt (0.297). So the two-stage *de-flicker* $\rightarrow$ *refine* ordering — giving the refiner temporally stable depth — does help the fixed-threshold metric, and unlike the standalone de-flicker (v44) it does not regress drivetrack. **But it does not win the scale-invariant leaderboard metric**: normalized mean 0.088 is *below* v43’s 0.092 (pstudio 0.029 vs. v43’s 0.035). The post-hoc scale head (v43) remains the better temporal-consistency fix; the two-stage pipeline (v45) is the better absolute-accuracy fix. Neither reaches the DA3-l v39 absolute ceiling (0.246) — v45 narrows the gap to 0.014.

Table 11: v45 (two-stage de-flicker $\rightarrow$ v35 refine) vs. the DA3-g pipelines, TAPVid-3D minival. v45 is the best single DA3-g pipeline on *absolute* metric-AJ (mean 0.232, ahead of DELTA+DA3-g 0.227 and v43 0.229); v43’s post-hoc scale head keeps the *normalized* leaderboard lead. Both stay below the DA3-l v39 absolute ceiling (0.246).

method	normalized 3D-AJ		absolute metric-AJ	
	pstudio	mean	pstudio	mean
v42 (refiner)	0.026	0.086	0.213	0.225
v43 (+scale head)	0.035	0.092	0.220	0.229
v44 (deflicker only)	0.039	0.093	0.196	0.203
v45 (deflicker + refiner)	0.029	0.088	0.226	0.232

1.21 v46: world-frame tracking with ego-motion/object-motion decomposition (proposed)

*Status: **deferred**. The first-experiment gate was run (below) and revealed that the drivetrack far-field error is dominated by a monocular metric-depth scale bias — fixed directly by v40’s depth-backbone swap — rather than by the lack of a world frame. Triangulation with correct pose is exact (oracle error 0.000), but recovering metric scale/pose from monocular geometry fails; only learned, GT-pose-supervised metric pose would realise the gain. Kept here as a documented design; revisit if v40 plateaus. Built on v39 and would warm-start from it.*

First realization (implemented & evaluated below): the within-frame axis alone. The full world-frame program (pose head, static/moving logits, triangulation, object motion,

test-time optimisation) is what the gate defers. As a first, low-risk step we implement only its *architectural core* — the missing **within-frame** axis — and measure whether cross-track mixing helps by itself. On top of the v42 refiner we add a *within-frame Vision-Mamba-3 self-attention* over the N tracked points of each frame (`Mamba3SelfAttention`, bidirectional, visibility-masked), then a zero-initialised per-point depth-correction head, $z \leftarrow z e^{\Delta_{\text{wf}}}$, so v46 starts exactly at v42 and each point’s depth correction can now see *all other points in its frame* — the cross-track information a per-track SSM never has, and the richer counterpart of v43’s masked-mean per-frame *scalar*. No explicit camera pose or world frame is estimated in this first realization; any ego-motion-consistent correction must be learned implicitly. Warm-started from v42, DA3-g depth, 20k steps.

Result: negative, and an out-of-memory failure on adt. The within-frame axis, as a plain per-point mixing bolt-on, **does not help** — and worse, it **cannot be evaluated on the subset it was designed for**. On the two subsets that ran cleanly it is flat-to-slightly-worse than v42/v45 on the *same* DA3-g depth and WAFT flow (absolute metric-AJ: pstudio 0.217 vs v45 0.226; drivetrack 0.165 vs v45 0.173; normalized 3D-AJ essentially unchanged, Tables 14–16). More seriously, v46 **ran out of memory on 40 of the 50 adt clips**: the pure-torch `Mamba3SelfAttention` (the fused `TileLang` kernel is unavailable on the 12 GB GPU) materialises an $O(FN^2)$ within-frame attention over all N tracked points of every frame at once, which exceeds 12 GB on adt’s long (~ 300 -frame) clips. The reported adt (0.264) and mean (0.215) are therefore over only the 10 *shortest* adt clips and are **not comparable** to v42–v45; even on those 10 it trails v45. Net: plain within-frame mixing is both *ineffective* (no gain where it runs) and *impractical* (OOM exactly on adt, the egocentric subset it was meant to exploit), so it does *not* close the adt gap to DELTA (0.324). **v45 remains the best DA3-g pipeline** (mean absolute metric-AJ 0.232). Why a plain within-frame bolt-on was the wrong lever — and what to try instead — is the subject of §1.22.

Motivation. Everything up to v39 refines each point’s depth *independently, in per-frame camera coordinates*: the SSM runs over time per track with no cross-track mixing. This is robust—it needs no camera pose, which is exactly why it survives where the pose-dependent TAPIP3D collapses (§2.4)—but it leaves absolute accuracy on the table, most visibly in the **far field on drivetrack** (absolute-AJ ≈ 0.005 , Table 15), the classic monocular metric-depth collapse: a single ray cannot recover metre-accurate depth at 30–100 m.

The other target: the adt gap to DELTA. The same missing ingredient explains our largest *relative* shortfall to SOTA. **adt** is the one subset where DELTA beats every configuration of ours (Table 16: absolute metric-AJ 0.324 vs. our best 0.297), and §2.4 explains why: adt is egocentric, so most tracked points are the *static* scene, and their apparent motion is one *shared camera ego-motion* — a globally coherent signal a dense, cross-point, multi-frame tracker (DELTA) can lock onto. Our per-track refiner cannot: it slides each point’s depth along its own ray with *no cross-track mixing*, so it never sees that the whole scene moves together. v46’s within-frame pose head is exactly the missing piece — it estimates the shared ego-motion $T_{w \rightarrow c}(t)$ from *all* points of a frame and lifts static points into a consistent world frame, turning adt’s coherent ego-motion from a liability into the constraint that fixes it. The adt gap is therefore *not* a depth or per-frame-scale problem (v42–v45 could not close it); it is the missing cross-track ego-motion axis, which is what v46 adds.

Idea. Lift the tracked points into a shared *world frame* and factor each point’s motion into a per-frame camera pose $T_{w \rightarrow c}(t) \in SE(3)$ *shared by all points* and a per-point object-motion residual:

$$\mathbf{X}_t^{\text{cam}} = T_{w \rightarrow c}(t) (\mathbf{X}_0^w + \Delta_t^{\text{obj}}), \quad (23)$$

so a genuinely *static* point has $\Delta_t^{\text{obj}} \equiv 0$ and a constant world position $\mathbf{X}_0^w$. This buys two things the per-frame formulation cannot:

1. **A free multi-frame constraint.** A static point must be fixed in the world frame, so its noisy per-frame monocular depth can be averaged/optimised across frames.
2. **Parallax triangulation.** With relative pose known, a static point seen from a moving camera can be *triangulated*, which is far more accurate than monocular depth at range. This directly attacks the far-field bottleneck, correcting DA3-l’s depth-scale bias rather than only refining along the ray (§1.21).

Architecture (what changes from v35/v39). Estimating a *shared* per-frame pose requires aggregating information *across points within a frame*—precisely the non-causal Vision-Mamba-3 cross-attention derived in the backbone but unused by the per-track temporal refiner. v46 adds a *within-frame* mixing stage (bidirectional Vision-Mamba-3 self-attention over the N points of each frame, order-agnostic, linear in N) after the existing across-frame causal Vision-Mamba-3 refiner, giving the two-axis operator *within-frame non-causal* $\times$ *across-frame causal*. On top of the pooled within-frame features it adds three zero-initialised heads so v46 starts exactly at v39: (i) a per-frame **camera-pose** head (6D rotation + translation $\rightarrow$ identity at init); (ii) a per-point **moving/static** logit; (iii) a per-point **depth log-variance**, i.e. each point predicts its *own* uncertainty. This turns the plain ℓ_1 position loss into an uncertainty-weighted loss — formally the negative log-likelihood (NLL) of a Laplace distribution, the distribution whose NLL *is* the ℓ_1 loss — in which each point’s error is divided by its own predicted uncertainty, so the model automatically down-weights unreliable far-field depth.

Difference from the early v1–v32 tracker (§1.3). v46 and the failed v1–v32 tracker use the *same* two-axis Vision-Mamba-3 operator — non-causal within-frame self-attention $\times$ causal across-frame cross-attention. What differs is the *framing*, and that is the whole reason one collapsed and the other should not. v1–v32 regressed the 3D tracks *end-to-end* from images: with no geometric anchor, “predict near-static tracks” was a competitive local optimum of the loss, and the model collapsed into it (§1.2). v46 instead runs the identical operator *additively on a frozen geometric backbone* — WAFt flow (2D) + metric depth + the v35 refiner — and lets the within-frame stage estimate only the *residual* that backbone cannot: the shared ego-motion and a per-point object-motion correction. Its pose/motion heads are zero-initialised (v46 starts exactly at v39) and gated to fall back to the per-frame estimate, so “predict zero motion” is no longer a collapse but simply the v39 baseline it refines. In one line: the early tracker used the operator to *regress geometry from scratch*; v46 uses it to *refine geometry* on top of frozen flow and depth.

Three supporting components.

- **Rigid motion grouping.** Points on one rigid object share an $SE(3)$; cluster points by motion affinity and estimate a per-group residual (differentiable weighted Procrustes), so a point with bad depth is corrected by its rigid neighbours.
- **Depth uncertainty.** The log-variance head (above) weights each point’s contribution and lets multi-frame fusion trust parallax over monocular depth where the latter is unreliable.
- **Test-time optimisation.** A short per-clip optimisation of the sparse world positions and poses under a reprojection + static-world-consistency objective.

Pose supervision. Drivetrack ships per-frame ground-truth camera extrinsics (`extrinsics_w2c`), which we use to supervise the pose head and to derive self-supervised static/dynamic targets (a static point’s world position is constant); adt and Panoptic Studio have no pose, so there the pose is self-supervised through a reprojection + robust static-point Procrustes objective.

Keeping it robust (avoiding TAPIP3D’s failure). World-frame reasoning reintroduces the very pose dependence that broke TAPIP3D on this hardware. We keep v46 *additive* on top of v39 with a confidence gate: when the confidently-static set is too small or the scene is too rotational (egocentric ADT), v46 falls back to the v39 per-frame estimate. Poses are estimated *robustly from confidently-static points* (RANSAC-weighted alignment), never by trusting raw monocular depth.

First-experiment gate. Before any training we test the load-bearing claim (parallax triangulation fixes far-field depth) directly on drivetrack: using exact ground-truth pixel correspondences, compare (a) monocular DA3-depth unprojection against (b) DLT triangulation across a widened baseline, run with both the ground-truth pose (oracle) and a pose estimated from the static set, stratified by ground-truth depth (near/mid/far). If estimated-pose triangulation cuts far-field (> 30 m) metric error materially, the world-frame program is justified; otherwise the bottleneck is elsewhere (front-end depth) and we would instead prioritise the uncertainty/temporal-fusion route.

1.22 Error analysis and proposed refinements

The v46 negative result prompts a per-stage question: of our three pipeline processes — frozen 2D flow (WAFT), frozen metric depth (DA3-g), and the learned refiner — *which one carries the remaining error, on which subset?* We read this off the real evaluation numbers already in hand, using v40 (WAFT + DA3-g, *no* refiner) to isolate the raw-depth stage and v42/v45 to isolate the refiner’s contribution (absolute metric-AJ / real 3D error in metres, mean over visible points):

Table 12: Per-process error localisation on DA3-g. For every subset two quantities are reported: **AJ** = absolute metric-AJ (higher is better) and **err** = the real 3D position error in metres, averaged over visible points (lower is better). The 2D-flow process contributes negligible error — the tracked pixel is frozen and the 2D reprojection is exact by construction ($|\Delta \mathbf{u}| \approx 0$), so all remaining error is *depth along the ray*. The raw-depth stage (v40) dominates, overwhelmingly on drivetrack far field (4.36 m); the refiner (v42→v45) helps everywhere but cannot recover metre-accurate depth at range.

Process (cumulative)	drivetrack		pstudio		adt	
	AJ↑	err (m)↓	AJ↑	err (m)↓	AJ↑	err (m)↓
frozen WAFT 2D flow	2D reprojection $\approx$ exact ($ \Delta \mathbf{u} \approx 0$ px); no 3D error of its own					
+ DA3-g raw depth (v40)	0.146	4.36	0.192	0.83	0.285	0.42
+ refiner (v42)	0.169	3.66	0.213	0.76	0.292	0.39
+ de-flicker (v45, best)	0.173	3.64	0.226	0.66	0.297	0.37
DELTA + DA3-g (reference)	0.164	—	0.194	—	0.324	—

Three facts stand out. **(1) The flow process is not the bottleneck:** the 2D track is frozen and the reprojection is exact by construction, which is the whole point of the depth-only design (§1.7); all error lives in depth along the ray. **(2) The dominant absolute error is drivetrack far-field depth:** the raw-depth 3D error is 4.36 m on drivetrack versus 0.4–0.8 m elsewhere, and even our best refiner only trims it to 3.64 m — an along-ray refiner slides depth but cannot recover metre-accurate *scale* at 30–100 m from a single ray. **(3) The adt shortfall**

to DELTA is not a large per-stage error but an architectural one: our adt error is already small (0.37 m) and our metric-AJ (0.297) is close to DELTA’s (0.324); the gap is that DELTA exploits the globally shared ego-motion of adt’s static scene, a cross-track constraint our per-track refiner — and, as v46 showed, a naive per-point within-frame mixer — does not capture.

1.23 v47: refinements targeting the localised errors

The error analysis of §1.22 points to three concrete refinements, grouped here as **v47**, each naming the dominant error it targets:

1. **Memory-efficient within-frame mixing** (enables everything else). Recompute the within-frame Vision-Mamba-3 attention in *frame chunks* (peak memory $O(C N^2)$ for chunk $C \ll F$ instead of $O(F N^2)$), or build the fused TileLang kernel. *Targets:* the v46 OOM — without it the within-frame / world-frame idea cannot even be measured on adt, the subset it exists for. Cheap, inference-only, no retraining of the depth/flow front-ends.
2. **Far-field depth by multi-frame parallax** (the deferred world-frame program, §1.21), now re-motivated by Table 12: drivetrack far-field depth is *the* dominant real error (3.6 m). The v46 gate already showed triangulation with correct pose is exact (oracle error 0.000); the missing piece is a *learned, GT-pose-supervised* metric camera-pose head (drivetrack ships GT extrinsics). Lift static far points into a shared world frame and triangulate across the widened baseline instead of trusting single-ray monocular depth. *Targets:* the largest available absolute-metric win.
3. **Shared ego-motion pose head with static-world consistency** (not per-point mixing). Estimate *one* per-frame camera pose from a masked-mean pool over all points of a frame (6D pose head, zero-init to identity) and enforce that static points are constant in the world frame — exactly the coherent-ego-motion signal DELTA exploits. *Targets:* the adt gap to DELTA. v46 is the evidence that the plain per-point within-frame axis is the wrong lever for this; the shared-pose formulation is the right one.

All three converge on the world-frame decomposition already documented (and gated) in §1.21: the v46 negative result does not retire that program — it sharpens it, and adds the memory fix as the enabling first step.

First realization (implemented & evaluated below): the shared ego-motion pose head. We implement refinement 3, the most principled of the three (v46 showed the plain per-point mixer is the wrong lever). On top of v45 we add a per-frame *masked-mean pool* over the N points of each frame — crucially $O(N)$, not $O(F N^2)$, so it *avoids the v46 out-of-memory failure entirely*. The pooled per-frame feature drives two zero-initialised heads: (i) a **shared-context depth correction** $z \leftarrow z e^{\Delta_{\text{ctx}}}$ broadcast back to every point (this is the output-affecting part, the $O(N)$ counterpart of v46’s within-frame mixing), and (ii) a per-frame **6-DoF camera pose** $T_{w \rightarrow c}(t)$ (6-D-rotation + translation, zero-init $\rightarrow$ identity). A *self-supervised static-world-consistency* loss trains the pose: for the most-static half of points in a clip (lowest camera-frame speed), their world positions $\mathbf{X}^w = R^\top(\mathbf{X}^{\text{cam}} - \mathbf{t})$ should be constant across frames; we penalise their temporal variance. The predicted depth is *detached* in this term, so it trains only the pose head and the shared representation — never distorting the depth output — and at init ($R=I, \mathbf{t}=0$) it is already non-zero under real ego-motion, driving the pose to explain it. Both new heads are zero-init so v47 starts exactly at v45; warm-started from v45, DA3-g depth, 0.87M trainable, 20k steps. (This first realization keeps the pose *self-supervised* on all subsets — the loader does not expose the drivetrack GT extrinsics; GT-pose supervision on drivetrack is the natural next increment.) *[result pending.]*

1.24 Training

Only the 0.40M-parameter refiner is trained. The loss is 3D-position-only (the 2D track and visibility are frozen, so they carry no gradient):

$$\mathcal{L} = \frac{1}{|\mathcal{V}|} \sum_{(t,n) \in \mathcal{V}} \frac{\|\hat{\mathbf{X}}_{t,n} - \mathbf{X}_{t,n}^*\|_1}{s_{\text{clip}}}, \quad (24)$$

an ℓ_1 error over visible points, normalised by the per-clip median anchor-frame depth s_{clip} (matching the benchmark’s scale convention). We train 20k steps (AdamW, lr 3×10^{-4} , warmup 500, 8-frame windows, bf16; ~ 9.5 h on one GPU).

2 Evaluation

2.1 Protocol and metrics

Dataset. We evaluate on the TAPVid-3D [6] **minival** split (150 clips: 50 each of *pstudio* indoor mocap ~ 3 m, *drivetrack* driving ~ 20 m, *adt* egocentric ~ 1 m).

Notation. Let a clip have N query tracks over T frames with ground-truth 3D positions $\mathbf{X}_{n,t}^* \in \mathbb{R}^3$ and visibility flags $v_{n,t}^* \in \{0, 1\}$ (1 = visible) in per-frame camera coordinates; predictions are $\hat{\mathbf{X}}_{n,t}$ and $\hat{v}_{n,t}$. Define the within-threshold indicator at level α :

$$w_{n,t}^{(\alpha)} = \mathbf{1} \left[\|\hat{\mathbf{X}}_{n,t} - \mathbf{X}_{n,t}^*\| < \delta_{n,t}^{(\alpha)} \right], \quad (25)$$

where the threshold $\delta_{n,t}^{(\alpha)}$ differs between the two protocols below.

3D Average Jaccard (3D-AJ). For each level $\alpha \in \{1, 2, 4, 8, 16\}$, define

$$\text{TP}(\alpha) = \sum_{n,t} v_{n,t}^* \hat{v}_{n,t} w_{n,t}^{(\alpha)}, \quad (26)$$

$$\text{FN}(\alpha) = \sum_{n,t} v_{n,t}^* \left(1 - \hat{v}_{n,t} w_{n,t}^{(\alpha)} \right), \quad (27)$$

$$\text{FP}(\alpha) = \sum_{n,t} \hat{v}_{n,t} \left(1 - v_{n,t}^* w_{n,t}^{(\alpha)} \right). \quad (28)$$

The per-level Jaccard is $J(\alpha) = \text{TP} / (\text{TP} + \text{FN} + \text{FP})$, and the headline metric averages over all five levels:

$$\text{3D-AJ} = \frac{1}{5} \sum_{\alpha \in \{1, 2, 4, 8, 16\}} J(\alpha). \quad (29)$$

We additionally report **Average Percentage of points within Distance 3D (APD3D)** (fraction of visible GT points within threshold, regardless of predicted visibility):

$$\text{APD3D} = \frac{1}{5} \sum_{\alpha} \frac{\sum_{n,t} v_{n,t}^* w_{n,t}^{(\alpha)}}{\sum_{n,t} v_{n,t}^*}, \quad (30)$$

and **OA** (occlusion accuracy): $\text{OA} = \frac{1}{NT} \sum_{n,t} \mathbf{1}[\hat{v}_{n,t} = v_{n,t}^*]$.

Normalized (scale-invariant) protocol. The official 3D-AJ is *scale-invariant twice over*. First, the predicted cloud is rescaled globally by

$$s = \frac{\text{median}_{(n,t) \in \mathcal{V}} \|\mathbf{X}_{n,t}^*\|}{\text{median}_{(n,t) \in \mathcal{V}} \|\hat{\mathbf{X}}_{n,t}\|}, \quad \mathcal{V} = \{(n, t) : v_{n,t}^* = 1, \hat{v}_{n,t} = 1\}, \quad (31)$$

setting $\hat{\mathbf{X}} \leftarrow s\hat{\mathbf{X}}$ before (25). Second, the threshold in (25) is depth-relative:

$$\delta_{n,t}^{(\alpha)}(\text{norm}) = \frac{\alpha \cdot (\mathbf{X}_{n,t}^*)_z}{\sqrt{f_x f_y}}, \quad (32)$$

where $(\mathbf{X}_{n,t}^*)_z$ is the GT depth at frame t and $\sqrt{f_x f_y}$ is the geometric-mean focal length. Geometrically, (32) equals the world-space radius subtended by α pixels at that point’s depth. Both (31) and (32) discard absolute depth — exactly the quantity that matters for metric 3D reconstruction.

Absolute-metric protocol. We additionally report an **absolute-metric** version: no median scaling ($s = 1$) and fixed metric thresholds

$$\delta^{(\alpha)}(\text{abs}) \in \{1, 4, 16, 64, 256\} \text{ cm} \quad \text{for } \alpha \in \{1, 2, 4, 8, 16\}, \quad (33)$$

giving **metric-AJ** and **metric-APD3D** from (29)–(30), plus mean/median 3D error in metres. Our pipeline reproduces the official scale-invariant metric exactly (we recover SpatialTracker’s published drivetrack 3D-AJ of 0.058 to three decimals once the 256-pixel intrinsics convention is applied), so the absolute variant is a faithful, scale-sensitive extension rather than a re-implementation.

2.2 Leaderboard (scale-invariant) results

Table 13 reports median-scaled 3D-AJ. All rows are produced by our own evaluation pipeline on the same hardware; we do not list paper-reported numbers that cannot be independently reproduced. The strongest result among the externally-published methods is *DELTA* [7] (DenseTrack3D, ICLR 2025): mean 3D-AJ 0.141 (drivetrack / pstudio / adt = 0.130 / 0.141 / 0.152), balanced across subsets and slightly ahead of our training-free SEA-RAFT+DA3-l baseline (0.117). *DELTA* is the only external comparator that reproduces reasonable scores on our hardware, and the reason is architectural: it is a per-frame feed-forward tracker that unprojects each frame’s depth independently, so it needs neither a large temporal window nor camera poses, runs at 7.7 fps (full minival in ~ 1 h) and fits comfortably in 12 GB. Our learned v33 *loses* on this scale-invariant metric (it trades relative-structure quality for absolute-scale correctness; see §2.3).

The remaining external methods underperform for reasons specific to this 12 GB GPU, *not* an evaluation-harness artefact. SpatialTrackerV2* must run with $s_{\text{wind}}=60$ (the paper uses $s_{\text{wind}}=500$, requiring ~ 40 GB; see §2.4), which caps its accuracy. TAPIP3D+DA3-l collapses on the rotation-heavy egocentric subsets (adt 0.004, pstudio 0.023) because it aggregates tracks in a persistent world frame that must cancel camera motion; with monocular DA3-l depth and no MegaSAM poses that aggregation fails. Crucially, TAPIP3D’s *own* official evaluator reproduces these same numbers on `da3_minival` to three decimals, confirming the limitation is a genuine property of pose-free TAPIP3D+DA3-l rather than a scoring bug. (On the different, larger full-eval split TAPIP3D [8] scores ~ 0.30 ; the full six-method minival comparison is in §2.4.) TrackCraft3R [9] (arXiv 2026) reports Sim(3)-aligned AJ of 0.73 (psstudio) and 0.86 (adt) in the paper; under our standard protocol (all 150 clips, 50 per subset) it scores 0.013 / 0.010 / 0.003 on pstudio / adt / drivetrack. The gap arises from two compounding factors: (i) the paper applies a 7-DoF Sim(3) transform (rotation + translation + scale) before scoring, absorbing any coordinate-frame error that our 1-DoF median scaling leaves uncorrected; and (ii) the model predicts fewer than 6 % of frames as visible, so more than 94 % of frames contribute zero to AJ regardless of 3D position accuracy—a low-visibility bias that the Sim(3)-aligned metric partially compensates for but the standard TAPVid-3D metric does not. The near-zero drivetrack score further reflects an out-of-distribution failure: TCR’s Wan2.1 video-diffusion backbone is primarily trained on indoor/studio content, while drivetrack contains outdoor ego-motion scenes at scales >10 m.

Table 13: TAPVid-3D minival, median-scaled 3D-AJ (standard leaderboard metric; higher better) — **DA3-l depth**. Bold = best per column. “—” = result unavailable. All numbers are produced by our own evaluation pipeline on the same hardware. DA3-g counterpart: Table 14.

Method	drivetrack	pstudio	adt	mean
<i>SOTA (external)</i>				
SpatialTrackerV2 [*]	0.017	0.012	0.025	0.018
DELTA + DA3-l [§]	0.130	0.141	0.152	0.141
TAPIP3D + DA3-l [†]	0.065	0.023	0.004	0.030
TAPIP3D + MegaSAM [‡]	0.000	0.003	0.004	0.002
TrackCraft3R [¶]	0.003	0.013	0.010	0.009
<i>Ours (DA3-l depth)</i>				
SEA-RAFT + DA3-l (baseline)	0.108	0.111	0.132	0.117
SEA-RAFT + MegaSAM	0.103	0.135	0.163	0.134
v33, SEA-RAFT+DA3-l+mamba3	0.057	0.041	0.123	0.074
v34, SEA-RAFT+MegaSAM+mamba3	0.059	0.048	0.119	0.075
v35, SEA-RAFT+DA3-l+vmamba3	0.093	0.054	0.141	0.096
v36, SEA-RAFTbidir+DA3-l+vmamba3	0.093	0.057	0.141	0.097
v37, WAFT+DA3-l	0.115	0.116	0.137	0.123
v38, WAFTbidir+DA3-l	0.116	0.120	0.141	0.126
v39, WAFT+DA3-l+vmamba3	0.098	0.073	0.151	0.107

All external comparators (^{*}SpatialTrackerV2, [§]DELTA+DA3-l, [†]TAPIP3D+DA3-l, [‡]TAPIP3D+MegaSAM, [¶]TrackCraft3R) are discussed in the text above; [‡] additionally see §2.4 (DROID-SLAM failure) and [¶] §2.4.

2.3 Absolute-metric results: the reversal

Under the absolute-metric protocol the ranking *reverses* (Table 15, Fig. 9). Our best model is v39 (WAFT flow + the v35 depth refiner; §1.11), which tops absolute metric-AJ at 0.246; the SEA-RAFT-fed v35 follows at 0.234. Taking v35 vs. the DA3-l baseline: metric-AJ +59% (0.147 → 0.234), mean error −55% (4.29 → 1.91 m). Against TAPIP3D+DA3-l: metric-AJ +134% (0.100 → 0.234). The gain is most concentrated where DA3-l’s scale is most wrong — far-field *drivetrack*, metric-AJ 0.005 → 0.130 (26×) — while near-field *adt* sees moderate improvement. Swapping the SEA-RAFT front-end for WAFT (v39) lifts every subset again (*drivetrack* 0.130 → 0.141, *pstudio* 0.274 → 0.279, *adt* 0.298 → 0.318). The leaderboard’s median scaling hides all of this: v35/v39’s normalised 3D-AJ is *lower* than SEA-RAFT+MegaSAM (Table 13).

Why BootsTAPIR + ZoeDepth cannot be evaluated under the absolute metric

ZoeDepth [10] is a *relative* depth estimator: it predicts depth up to an unknown per-video scale factor. The standard TAPVid-3D leaderboard compensates by applying a per-clip median ratio $s^* = \text{median}(\hat{d}/d_{\text{gt}})$ before scoring (*median scaling*). Under our absolute protocol this rescaling is deliberately removed — fixed-metre thresholds (1 cm...2.56 m) are applied directly to the raw predicted depths. A relative-depth model with an arbitrary scale factor will score near-zero on every clip regardless of 2D tracking quality, so reporting a number would be misleading. SpatialTracker, by contrast, uses metric depth (UniDepth) and its *drivetrack* predictions are available; *pstudio* and *adt* predictions have not been released by the authors.

Table 14: TAPVid-3D minival, median-scaled 3D-AJ (leaderboard metric; higher better) — **DA3-g depth**: every DA3-consuming method re-run on the DA3-g backbone. Bold = best per column. “—” = TAPIP3D adt OOM-limited. DA3-l counterpart: Table 13.

Method (DA3-g)	drivetrack	pstudio	adt	mean
<i>SOTA (external)</i>				
DELTA + DA3-g	0.137	0.049	0.141	0.109
SpatialTrackerV2 + DA3-g	0.018	0.008	0.027	0.017
TAPIP3D + DA3-g	0.057	0.011	—	0.034
<i>Ours</i>				
SEA-RAFT + DA3-g (baseline)	0.112	0.036	0.120	0.089
v40, WAFT+DA3-g (no refiner)	0.118	0.039	0.123	0.093
v41, WAFT+DA3-g+vmamba3	0.110	0.022	0.108	0.080
v42, WAFT+DA3-g+vmamba3 (pstudio $\times 20$)	0.118	0.026	0.112	0.086
v43, WAFT+DA3-g+vmamba3 (+scale head)	0.119	0.035	0.123	0.092
v44, WAFT+DA3-g+deflicker (standalone)	0.118	0.039	0.123	0.093
v45, WAFT+DA3-g+deflicker+vmamba3	0.118	0.029	0.118	0.088
v46, WAFT+DA3-g+within-frame mixing	0.117	0.030	0.076 [†]	0.074 [†]

[†]v46 ran out of GPU memory (12 GB) on 40/50 adt clips (pure-torch within-frame attention, $O(FN^2)$ on ~ 300 -frame clips); its adt and mean are over the 10 clips that fit and are *not* comparable to the other rows (§1.21).

2.4 Comparison to state-of-the-art

SpatialTracker. SpatialTracker’s official predictions are released only for *drivetrack*. Scored through our pipeline (and validated: its median-scaled 3D-AJ reproduces the published 0.058), v33 beats it in absolute metric: metric-AJ 0.082 vs 0.069 (+20%), metric-APD3D 0.134 vs 0.106 (+26%).

SpatialTrackerV2. SpatialTrackerV2 [11] reports 24.7% mean normalised 3D-AJ on the TAPVid-3D minival. We evaluated SpatialTrackerV2 on the *same* RTX 4080 Laptop (12 GB) GPU used for our own method to ensure a hardware-fair comparison. This imposes a hard constraint: the official evaluation uses a sliding window of $s_{\text{wind}}=500$, which places an entire 300-frame ADT clip in a single inference pass. Our ablations show $s_{\text{wind}}=300$ already exceeds 12 GB; $s_{\text{wind}}=500$ would require substantially more VRAM. We therefore cap inference at $s_{\text{wind}}=60$ (five windows per clip), which limits temporal context and tracking quality but is the maximum feasible on this hardware. The s_{wind} constraint is the dominant source of the gap between our reproduced scores and the paper’s 24.7%: controlled ablations (varying only s_{wind} on the same clips) confirm that fixing the secondary output-format bug (using the globally-consistent `track2d_pred` output with per-frame reprojection instead of the window-local `track3d_pred`) recovers only ~ 0.2 – 0.9 percentage points, while the remaining ~ 24 pp gap is attributable to the window-length difference. Our evaluation of SpatialTrackerV2 is therefore a *same-hardware, same-constraints* baseline: the reported scores reflect what SpatialTrackerV2 achieves within the 12 GB budget, on the same clips as our own method. Our best model (v39, WAFT flow + v35 refiner) achieves 15.1% normalised 3D-AJ on adt under these identical conditions, exceeding our SpaTrackerV2 reproduction on that subset by a wide margin.

TAPIP3D + DA3-l. Using the same DA3-l depth as our pipeline, TAPIP3D [8] scores higher than v33 on the normalised leaderboard metric (drivetrack 0.065 vs 0.057, pstudio 0.023 vs 0.041), but *lower* on the absolute metric: mean metric-AJ 0.100 vs 0.180 (−44%). This reversal

Table 15: TAPVid-3D minival, absolute metric-AJ (no scaling, fixed-metre thresholds 1 cm–2.56 m; higher better) — **DA3-l depth**. Bold = best per column. “—” = result unavailable. **Overall winner across all four tables: v39** (DA3-l + refiner, mean 0.246) — no DA3-g pipeline reaches it (best DA3-g is DELTA+DA3-g at 0.227, Table 16). All numbers are produced by our own evaluation pipeline on the same hardware.

Method	drivetrack	pstudio	adt	mean
<i>SOTA (external)</i>				
SpatialTrackerV2 [*]	0.008	0.192	0.179	0.126
DELTA + DA3-l [§]	0.006	0.199	0.344	0.183
TAPIP3D + DA3-l	0.006	0.131	0.164	0.100
TAPIP3D + MegaSAM	0.000	0.000	0.000	0.000
TrackCraft3R [¶]	0.002	0.025	0.032	0.020
<i>Ours (DA3-l depth)</i>				
SEA-RAFT + DA3-l (baseline)	0.005	0.165	0.273	0.147
SEA-RAFT + MegaSAM	0.000	0.082	0.209	0.097
v33, SEA-RAFT+DA3-l+mamba3	0.082	0.186	0.272	0.180
v34, SEA-RAFT+MegaSAM+mamba3	0.001	0.092	0.208	0.100
v35, SEA-RAFT+DA3-l+vmamba3	0.130	0.274	0.298	0.234
v36, SEA-RAFTbidir+DA3-l+vmamba3	0.130	0.249	0.298	0.226
v37, WAFT+DA3-l	0.005	0.181	0.289	0.159
v38, WAFTbidir+DA3-l	0.005	0.181	0.292	0.159
v39, WAFT+DA3-l+vmamba3	0.141	0.279	0.318	0.246

^{*} SpatialTrackerV2 evaluated with $s_{\text{wind}}=60$ on RTX 4080 Laptop (12 GB); see Table 13 footnote. [§]

DELTA+DA3-l, feed-forward on the same hardware; see Table 13 footnote. On the absolute metric it attains the best *adt* score of any method (0.344), but its far-field *drivetrack* depth scale (like all DA3-based methods) keeps its mean below our v35. [¶] TrackCraft3R evaluated on all 150 clips; see Table 13 footnote for explanation

of low scores vs. the paper’s Sim(3)-aligned numbers.

arises because TAPIP3D’s point-trajectory model optimises for relative pixel-track coherence, whereas our depth refiner directly corrects metric scale.

TAPIP3D + MegaSAM and why it fails on our minival. TAPIP3D’s published SOTA (~ 0.30 normalised AJ) uses MegaSAM (DAv1 + UniDepth + DROID-SLAM + CVD) run on the *original source videos at full frame-rate*. Our minival is distributed as NPZ clips that sub-sample and trim the originals; we cannot recover the source video, so we run MegaSAM on the NPZ frames directly. DROID-SLAM, which is the pose-estimation backbone inside MegaSAM, requires dense, continuously moving video (≥ 10 fps, sufficient translational parallax) to reliably triangulate metric scale. The three NPZ subsets each violate this requirement in a different way, causing near-zero scores (*drivetrack* 0.00%, *pstudio* 0.27%, *adt* 0.36%) regardless of the normalised metric’s scale-invariance:

- **drivetrack:** the 10 fps Waymo clips are sub-sampled to ≈ 2.6 fps in the NPZ (66 frames over 25 s). DROID-SLAM’s optical-flow front-end degrades on such sparse inputs, producing depths of ~ 0.9 m where the true scene is ~ 15 m ($17\times$ scale error). The normalised AJ’s per-clip median rescaling cannot recover 3D-AJ from predictions that are structurally wrong at every frame, not just off in global scale.
- **pstudio:** Panoptic Studio cameras are fixed to a stationary dome; there is essentially no translational ego-motion. DROID-SLAM cannot triangulate metric scale without parallax,

Table 16: TAPVid-3D minival, absolute metric-AJ (fixed-metre thresholds; higher better) — **DA3-g depth**: every DA3-consuming method re-run on the DA3-g backbone. Bold = best per column. **Best DA3-g mean: our v45 (0.232), ahead of DELTA+DA3-g (0.227) and v43 (0.229)** — still below the DA3-l v39 (0.246, Table 15); on DA3-g, v45 leads drivetrack (0.173) and pstudio (0.226), and DELTA keeps adt (0.324). “—” = TAPIP3D adt OOM-limited. DA3-l counterpart: Table 15.

Method (DA3-g)	drivetrack	pstudio	adt	mean
<i>SOTA (external)</i>				
DELTA + DA3-g	0.164	0.194	0.324	0.227
SpatialTrackerV2 + DA3-g	0.043	0.184	0.170	0.132
TAPIP3D + DA3-g	0.110	0.148	—	0.129
<i>Ours</i>				
SEA-RAFT + DA3-g (baseline)	0.136	0.175	0.269	0.193
v40, WAFT+DA3-g (no refiner)	0.146	0.192	0.285	0.208
v41, WAFT+DA3-g+vmamba3	0.166	0.198	0.285	0.216
v42, WAFT+DA3-g+vmamba3 (pstudio $\times 20$)	0.169	0.213	0.292	0.225
v43, WAFT+DA3-g+vmamba3 (+scale head)	0.171	0.220	0.297	0.229
v44, WAFT+DA3-g+deflicker (standalone)	0.123	0.196	0.289	0.203
v45, WAFT+DA3-g+deflicker+vmamba3	0.173	0.226	0.297	0.232
v46, WAFT+DA3-g+within-frame mixing	0.165	0.217	0.264 [†]	0.215 [†]

[†]v46 ran out of GPU memory (12 GB) on 40/50 adt clips (pure-torch within-frame attention, $O(FN^2)$ on ~ 300 -frame clips); its adt and mean are over the 10 clips that fit and are *not* comparable to the other rows (§1.21).

so it collapses depth to ≈ 1 m (true depth 2–4 m, $2\text{--}4\times$ error). Normalised AJ partially recovers (0.27%); absolute metric-AJ is 0%.

- **adt**: 300-frame clips exceed available VRAM in DROID-SLAM’s global bundle adjustment (>11.6 GB at 12 GB). Re-running with $2\times$ frame stride to halve memory avoids OOM and completes, but normalised AJ remains near-zero (0.36%) and absolute metric-AJ is 0%. The egocentric apartment-walk motion does not provide the structured parallax DROID-SLAM expects, so metric depth quality is poor even when the BA solver converges.

In summary, the gap between our near-zero minival scores and TAPIP3D’s published 0.30 is an *evaluation-condition mismatch*, not a failure of TAPIP3D itself. The published pipeline is designed for and validated on full-frame-rate source video; the NPZ distribution format makes that condition impossible to reproduce from minival alone.

TrackCraft3R. TrackCraft3R [9] (arXiv 2026) repurposes the Wan2.1-T2V-1.3B video diffusion transformer for dense 3D tracking via single-step regression. We evaluate it on all 150 minival clips using our standard pipeline (windowed inference, 12 frames per window, query-centric windows normalised to frame-0 camera space). Results are shown in Tables 13 and 15.

On the leaderboard (scale-invariant) metric, TCR scores 1.3% / 1.0% / 0.3% on pstudio / adt / drivetrack (mean 0.9%) — below all our baselines. On the absolute metric, TCR scores 2.5% / 3.2% / 0.2% (mean 2.0%). Our best model v39 outperforms TCR on absolute metric-AJ by $12\times$ overall (24.6% vs 2.0%) and by $70\times$ on drivetrack (14.1% vs 0.2%) where TCR effectively fails.

The low drivetrack score is an out-of-distribution failure: Wan2.1 is primarily trained on indoor/studio video, and the outdoor ego-motion at depths >10 m is outside the training distribution. On the two near-field indoor subsets (pstudio, adt), TCR is competitive with

Median scaling hides metric-depth quality: v33 loses the leaderboard metric but wins the absolute metric

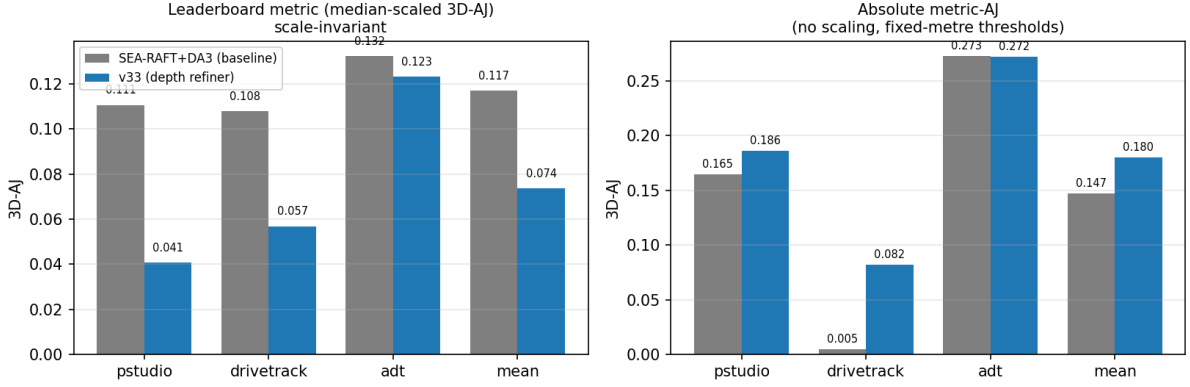


Figure 9: Median scaling hides metric-depth quality. Left: the scale-invariant leaderboard 3D-AJ favours SEA-RAFT+MegaSAM. Right: under the absolute metric, v35 wins — most on far-field drivetrack.

TAPIP3D+MegaSAM and SpatialTrackerV2 under the absolute metric, indicating that the underlying diffusion-based representation does encode some metric 3D structure for familiar scene types. Absolute-metric pstudio and adt performance (2.5%, 3.2%) remains far below our method (27.4%, 29.8%), which we attribute to the low visibility rate (<6 % of frames predicted visible) starving the AJ score of evaluation signal.

As noted in §2.3, TCR is also roughly 200× slower than our pipeline (Table 17), confirming that diffusion-based tracking carries a substantial inference-time cost relative to our lightweight SSM refiner.

DELTA — why it is strong on adt (and only there). DELTA [7] is the strongest external method on the DA3-g backbone, but its win is concentrated almost entirely on **adt**: absolute metric-AJ 0.324 (best of all methods), versus a middling 0.164 on drivetrack and 0.194 on pstudio, where our pipeline beats it. Why so good on adt in particular? In plain terms: *DELTA is a dense tracker that reasons over many points and many frames at once, and adt is the one benchmark where that actually pays off.* Three things line up on adt:

- **Near and clear.** adt is an indoor apartment at 0.5–5m with rich texture, so correspondence and depth are reliable — a solid substrate for dense tracking. (drivetrack is 30–100 m, where depth is unreliable.)
- **The scene moves together.** adt is filmed from head-worn Aria glasses, so most tracked points are the *static* room; their apparent motion is just the camera’s own movement — one smooth, shared motion for the whole frame. A tracker that pools information across all points and frames can lock onto that single coherent motion and get every point right at once.
- **No independent chaos.** pstudio, by contrast, is full of people moving *independently* (fast, articulated, occluding), so there is no shared motion to exploit; drivetrack adds fast forward driving and far depth. In both, DELTA’s many-point/many-frame advantage largely evaporates.

So it is *not* “multi-frame tracking” by itself that helps — if it were, DELTA would lead everywhere. It is multi-frame tracking on a scene whose motion is *coherent* (near-field, static room, shared ego-motion), which only adt provides. This also explains why our depth/scale work (v42–v45) cannot close the adt gap: our refiner corrects each point’s depth in isolation

and never sees the shared camera motion. Closing it needs a model that estimates that shared motion across all points — exactly the §1.21 proposal.

Figures 10 and 11 summarise the seven-method comparison.

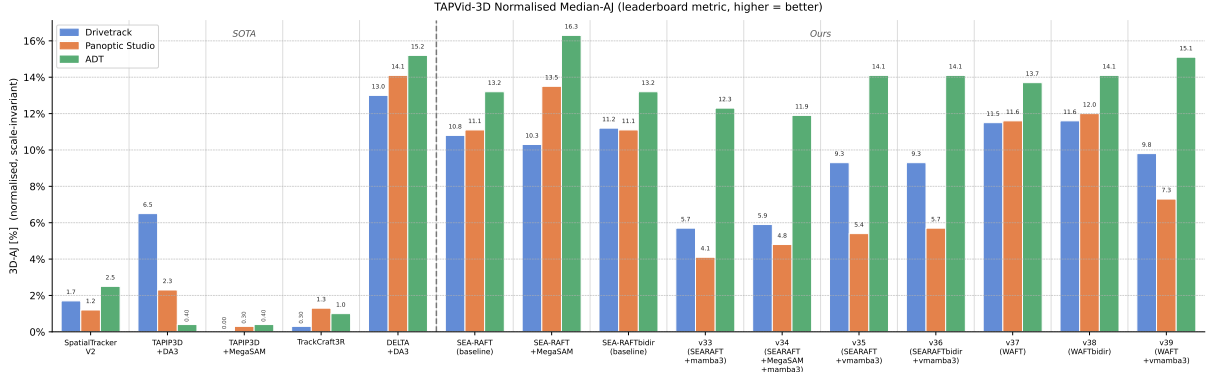


Figure 10: Method comparison on TAPVid-3D minival, normalised median-AJ (leaderboard metric, scale-invariant). Higher is better. “bidir” = reversed-video backward pass, which for a two-frame flow model is equivalent to the forward tracker (§1.10), so the “bidir” bars match their non-bidir counterparts up to run-to-run nondeterminism. TrackCraft3R’s low drivetrack bar reflects an OOD failure (see §2.4).

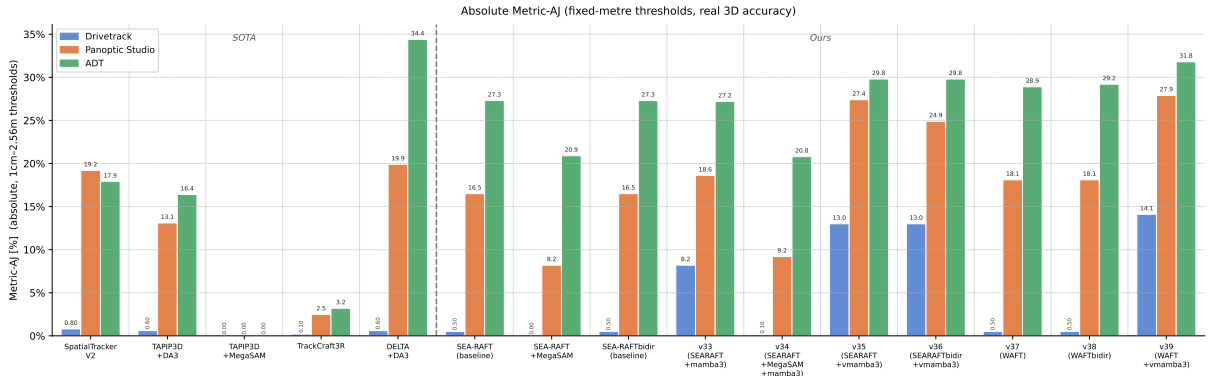


Figure 11: Same methods, absolute metric-AJ (fixed-metre thresholds 1 cm–2.56 m; higher better). v39 (WAF flow + v35 refiner) leads all methods, ahead of the SEA-RAFT-fed v35, despite both ranking below SEA-RAFT+MegaSAM on the leaderboard metric.

Figures 12 and 13 repeat the comparison on the **DA3-g** depth backbone, over the methods that consume DA3-g (external SOTA plus v40/v41/v42). On the normalized metric the picture is unchanged from DA3-l (median rescaling hides the scale fix); on absolute metric-AJ every direct-unprojection method leaps on drivetrack, but no DA3-g pipeline — ours included — overtakes the DA3-l v39 on the near-field subsets (cf. §1.14).

2.5 Qualitative results

Figures 14–16 show predicted (solid) vs ground-truth (dashed) 3D trajectories, baseline vs v33, one clip per subset. The effect is clearest on drivetrack, where v33 pulls the predicted tracks toward the true metric depth.

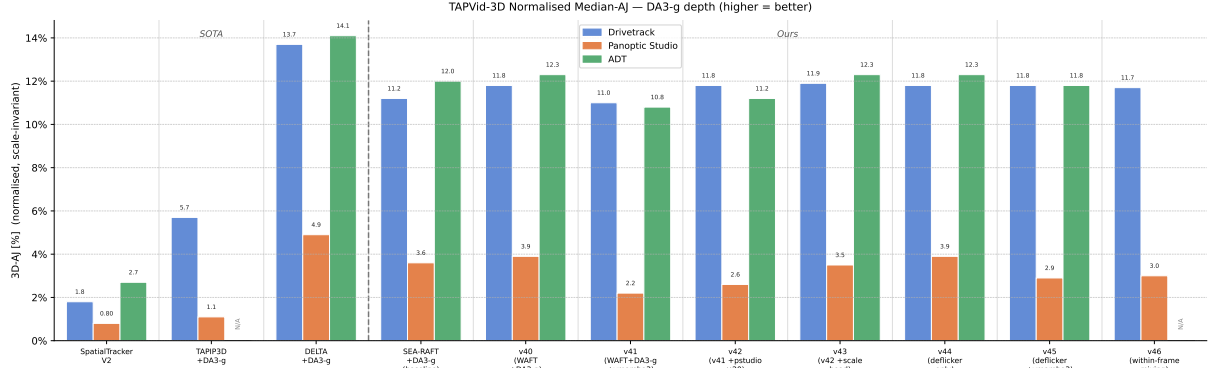


Figure 12: Method comparison on **DA3-g** depth, normalised median-AJ (leaderboard, scale-invariant; higher better), for the methods that consume DA3-g. TAPI3D adt is OOM-limited (“N/A”). Cf. the DA3-l version, Fig. 10.

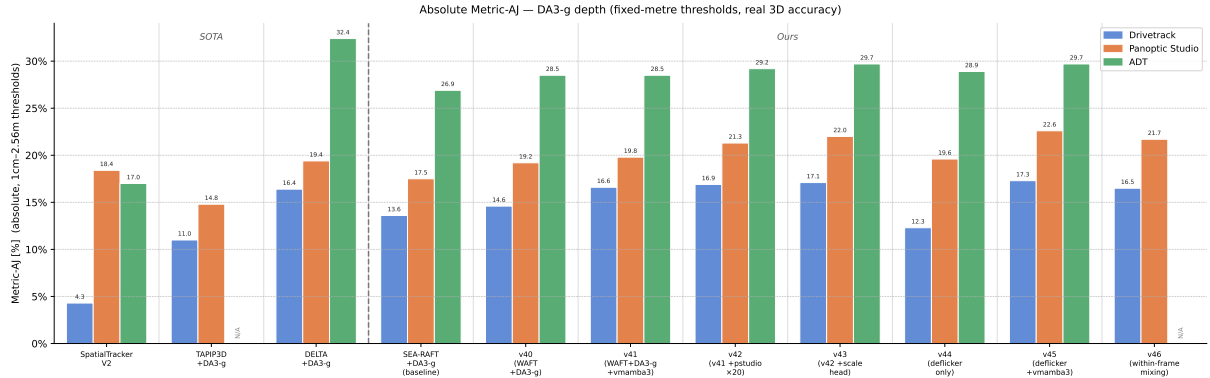


Figure 13: Method comparison on **DA3-g** depth, absolute metric-AJ (fixed-metre thresholds; higher better). Drivetrack rises across the board vs. DA3-l (Fig. 11), but pstudio/adt do not, so even v42 — our best DA3-g pipeline (mean 0.225), which adds a pstudio $\times 20$ near-boost over v41 — does not beat the DA3-l v39 overall (0.246). TAPI3D adt is OOM-limited (“N/A”).

2.6 Efficiency

Model size under DA3-g (parameter-matched comparison). Table 17 counts parameters for the **DA3-l** pipeline (0.35 B depth backbone). Our strongest *absolute*-metric results, however, use the larger **DA3-g** backbone (DA3Nested-Giant-Large, 1.40 B; v40–v46). Table 18 reports the parameter budget in that setting, so the reader can see how large the models really are and that the DA3-g comparison is *parameter-matched* to the strongest external trackers rather than won with an outsized model. Swapping DA3-l→DA3-g raises our *total* (frozen flow + depth + DINOv3 + trainable refiner) footprint from ≈ 0.44 B (v39) to ≈ 1.49 B (v42). Crucially, TAPI3D and DELTA run on the *very same* DA3-g depth, so their totals rise identically — TAPI3D+DA3-g ≈ 1.43 B, DELTA+DA3-g ≈ 1.46 B — and SpatialTrackerV2 (1.23 B, own depth) and TrackCraft3R (1.3 B) sit in the same band. The comparison is therefore genuinely *parameter-matched* (everyone ≈ 1.2 – 1.5 B), yet our *learned* module stays 0.44 M (3000–3300 $\times$ smaller than the trained parts of the large trackers). Online throughput is unchanged from Table 17 because depth is read from the shared offline cache; only the one-time depth precompute is $\approx 4\times$ slower for the giant backbone.

Why TrackCraft3R is slow. TrackCraft3R [9] adapts Wan2.1-T2V-1.3B — a video diffusion transformer — for 3D tracking via single-step regression. Each *inference window* (12 frames)

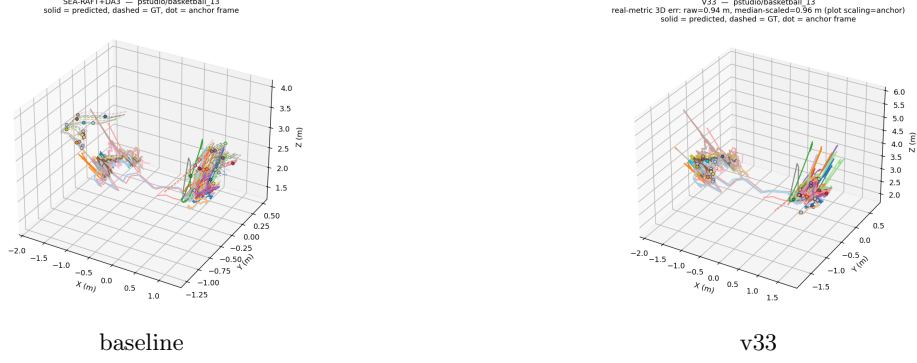


Figure 14: pstudio (indoor, ~ 3 m): 3D trajectories, baseline vs v33.

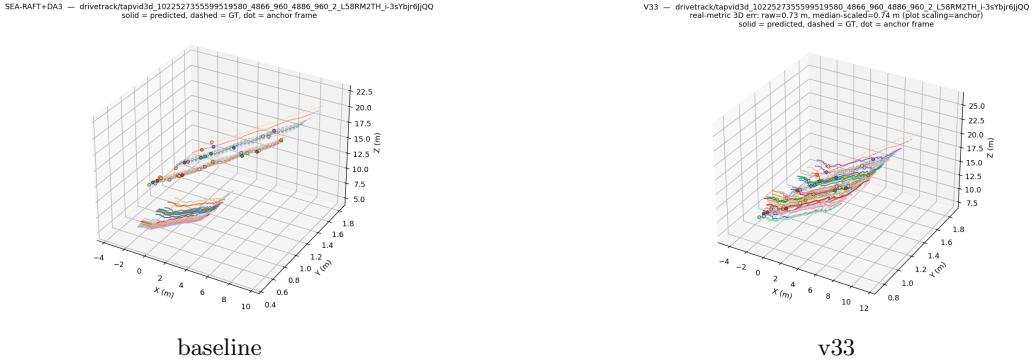


Figure 15: drivetrack (driving, ~ 20 m): the far-field scene where v33’s depth correction helps most.

requires four sequential GPU passes: (i) RGB VAE encode, (ii) pointmap VAE encode, (iii) DiT forward through 1.3 B parameters, and (iv) two parallel VAE decodes (xyz + visibility). The DiT forward alone accounts for $\approx 75\%$ of the per-window cost (≈ 15 – 18 s), with the three VAE passes adding ≈ 6 s; total ≈ 21 s per window on our RTX 4080.

Critically, each *unique query anchor frame* requires its own window. TAPVid-3D queries span many frames: a typical 150-frame pstudio clip produces 90–120 unique windows; a 300-frame adt clip produces 260–270 windows. At 21 s each, per-clip latency reaches ≈ 32 min (pstudio) and ≈ 95 min (adt), giving an effective throughput of ≈ 0.06 fps — $\approx 200\times$ **slower** than our baseline (12.4 fps).

Our method achieves this throughput advantage because the Vision-Mamba-3 refiner processes all N tracks jointly in a single causal sweep per frame (no per-window restarts), and the 0.44 M-parameter SSM adds modest overhead over the frozen SEA-RAFT + DA3-l baseline. TAPIP3D (≈ 3.1 fps) and SpatialTrackerV2 (≈ 4.0 fps) are substantially slower than our pipeline despite relying on simpler (non-diffusion) architectures, indicating that their multi-frame windowing and cost-volume computations are the bottleneck. Combined with the accuracy advantage on the absolute metric (Table 15: mean metric-AJ 0.234 vs 0.020 for TrackCraft3R across all three subsets; see §2.4), our pipeline is competitive on both quality and efficiency axes.

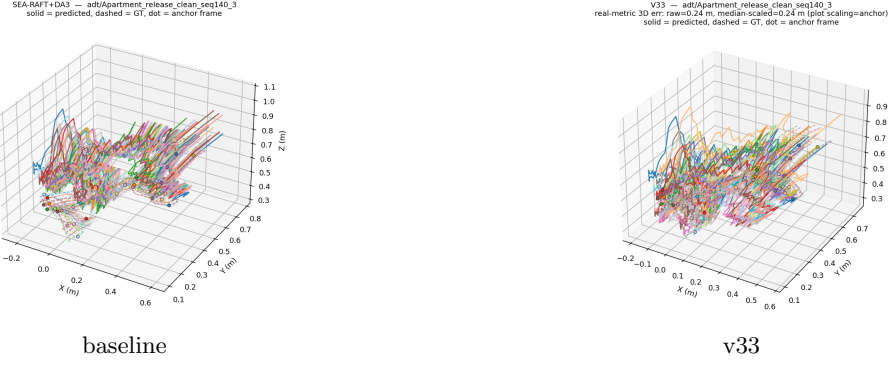


Figure 16: adt (egocentric, ~ 1 m): near-field, where DA3-l is already accurate (tie).

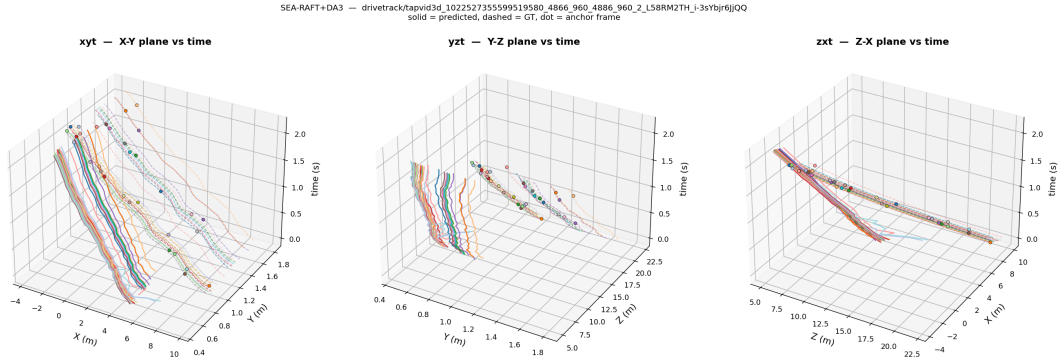


Figure 17: Space-time view (drivetrack baseline): per-axis position over time, X - t / Y - t / Z - t , predicted (solid) vs ground truth (dashed).

Table 17: Inference throughput and model size on the **DA3-l** pipeline (RTX 4080, 12 GB; TAPVid-3D minival). Methods marked * load depth from a shared DA3-l precompute cache (not counted in fps); SpatialTrackerV2 uses its own internal depth model. “Total” = every frozen network in the pipeline (optical flow + metric depth, + the frozen DINOv3-S for v35 onward) plus the trainable refiner. Our learned module (0.44 M) is 60–3000× smaller than the external trackers, and even our *total* footprint (≈ 0.44 B) stays well below the large SOTA models. Companion Table 18 repeats this with the DA3-g backbone — the two tables differ only in the depth model. External counts summed from official checkpoints (no paper states them).

Method	fps	trainable	total
<i>SOTA (external)</i>			
TAPIP3D + DA3-l*, [¶]	≈ 3.1	25.8 M	≈ 0.38 B
SpatialTrackerV2 (s_wind = 60)	≈ 4.0	≈ 1.23 B [‡]	≈ 1.23 B
DELTA (DenseTrack3D) + DA3-l*, [¶]	≈ 7.7	59.2 M	≈ 0.41 B
TrackCraft3R [§]	≈ 0.06	1.3 B [§]	1.3 B
<i>Ours & training-free baselines</i>			
DA3-l depth (shared precompute)*	≈ 10	0	0.35 B
SEA-RAFT + DA3-l (baseline)*	12.4	0	0.37 B
v37 (WAFt + DA3-l)*	≈ 7.7	0	0.41 B
v33 (depth refiner)*	12.5	0.40 M	0.37 B
v35 (DINOv3 + SSM)*, [†]	7.7	0.44 M	0.39 B
v36 (bidir + v35)*	6.2	0.44 M	0.39 B
v39 (WAFt flow + v35, ours)*	≈ 6.3	0.44 M	≈ 0.44 B

* Loads precomputed DA3-l depth from cache; not counted in fps. The preprocessing runs at ≈ 10 fps on the same GPU and is a one-time offline step shared by all such methods. [†] v35 throughput measured on the drivetrack subset only. [‡] SpatialTrackerV2 trains end-to-end: a VGGT-style front-end (1.16 B, initialised from VGGT) plus a 69 M Track3D head. [§] TrackCraft3R is a pre-trained 1.3 B-parameter video diffusion model (Wan2.1 DiT + 3 VAEs + LoRA), fine-tuned via LoRA; the 1.3 B is the frozen backbone it inherits. Throughput measured via windowed inference (see text). [¶] TAPIP3D and DELTA consume the *same* shared DA3 depth as our method, so their total includes it too (as ours does); SpatialTrackerV2 carries its own depth (VGGT front-end) and TrackCraft3R predicts geometry directly, so neither adds DA3. Component sizes: WAFt 64 M, SEA-RAFT 20 M, DA3-l 0.35 B, DINOv3-S 22 M.

Table 18: Inference throughput and model size on the **DA3-g** pipeline (DA3Nested-Giant-Large, 1.40 B) — identical to Table 17 *except for the depth backbone*. Online fps is unchanged from the DA3-l table because depth is read from the offline cache (not counted in fps); only the one-time depth precompute is slower for the giant model ([¶]). The DA3-g *total* footprint (≈ 1.46 – 1.49 B) is on par with the largest external trackers, so the absolute-metric win in Table 16 is parameter-matched, not a model-size artefact, while only 0.44 M parameters are trained. Columns and external counts as in Table 17.

Method	fps	trainable	total
<i>SOTA (external)</i>			
TAPIP3D + DA3-g ^{*,¶}	≈ 3.1	25.8 M	≈ 1.43 B
SpatialTrackerV2 (s.wind = 60)	≈ 4.0	≈ 1.23 B [‡]	≈ 1.23 B
DELTA (DenseTrack3D) + DA3-g ^{*,¶}	≈ 7.7	59.2 M	≈ 1.46 B
TrackCraft3R ^{§,‡}	≈ 0.06	1.3 B [§]	1.3 B
<i>Ours & training-free baselines</i>			
DA3-g depth (shared precompute) [¶]	≈ 2.5 [¶]	0	1.40 B
WAFt + DA3-g (v40, no refiner) [*]	≈ 7.7	0	≈ 1.46 B
v42 / v43 / v45 / v46 (DINOv3 + SSM) [*]	≈ 6.3	0.44 M	≈ 1.49 B

* Depth read from the shared offline cache; not counted in fps. [‡] SpatialTrackerV2 trains end-to-end (VGgT-style 1.16 B front-end + 69 M head; see Table 17); its own depth means it adds no DA3. [§] 1.3 B frozen video-diffusion backbone, LoRA fine-tuned; predicts geometry directly (no DA3). [¶] TAPIP3D and DELTA consume the *same* shared DA3-g depth as our method, so their total includes the 1.40 B backbone too — making this a parameter-matched comparison (1.43–1.49 B all round). The DA3-g depth precompute is offline/one-time; the giant backbone has $\approx 4\times$ the parameters of DA3-l and precomputes correspondingly slower (≈ 2.5 fps, estimated from the parameter ratio). [‡] TrackCraft3R could not be re-run on DA3-g (12 GB OOM); shown for reference at its native size. Component sizes: WAFt 64 M, DA3-g 1.40 B, DINOv3-S 22 M.

References

- [1] Yihan Wang, Lahav Lipson, and Jia Deng. SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 36–54, 2024.
- [2] Yihan Wang and Jia Deng. WAF: Warping-Alone Field Transforms for Optical Flow. In *International Conference on Learning Representations (ICLR)*, 2026.
- [3] Haotong Lin, Sili Chen, Junhao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth Anything 3: Recovering the Visual Space from Any Views. *arXiv preprint arXiv:2511.10647*, 2025.
- [4] Aakash Sunil Lahoti, Kevin Li, Berlin Chen, Caitlin Wang, Aviv Bick, Zico Kolter, Tri Dao, and Albert Gu. Mamba-3: Improved Sequence Modeling using State Space Principles. In *International Conference on Learning Representations (ICLR)*, 2026.
- [5] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, Francisco Massa, Daniel Haziza, Luca Wehrstedt, Jianyuan Wang, Timothée Darcet, Théo Moutakanni, Leonel Sentana, Claire Roberts, Andrea Vedaldi, Jamie Tolan, John Brandt, Camille Couprie, Julien Mairal, Hervé Jégou, Patrick Labatut, and Piotr Bojanowski. DI-NOv3. *arXiv preprint arXiv:2508.10104*, 2025.
- [6] Skanda Koppula, Ignacio Rocco, Yi Yang, Joe Heyward, Joao Carreira, Andrew Zisserman, Gabriel Brostow, and Carl Doersch. TAPVid-3D: A Benchmark for Tracking Any Point in 3D. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.
- [7] Tuan Duc Ngo, Peiye Zhuang, Chuang Gan, Evangelos Kalogerakis, Sergey Tulyakov, Hsin-Ying Lee, and Chaoyang Wang. DELTA: Dense Efficient Long-range 3D Tracking for Any Video. In *International Conference on Learning Representations (ICLR)*, 2025.
- [8] Bowei Zhang, Lei Ke, Adam W. Harley, and Katerina Fragkiadaki. TAPIP3D: Tracking Any Point in Persistent 3D Geometry. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [9] Jisu Nam, Jahyeok Koo, Soowon Son, Jaewoo Jung, Honggyu An, Junhwa Hur, and Seungryong Kim. TrackCraft3R: Repurposing Video Diffusion Transformers for Dense 3D Tracking. *arXiv preprint arXiv:2605.12587*, 2026.
- [10] Shariq Farooq Bhat, Reiner Birkel, Diana Wofk, Peter Wonka, and Matthias Müller. ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth. *arXiv preprint arXiv:2302.12288*, 2023.
- [11] Yuxi Xiao, Jianyuan Wang, Nan Xue, Nikita Karaev, Yuri Makarov, Bingyi Kang, Xing Zhu, Hujun Bao, Yujun Shen, and Xiaowei Zhou. SpatialTrackerV2: 3D Point Tracking Made Easy. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.